



X ARTICLE · SYNTHETIC DATA

Recursive Synthetic Improvement

Zafir Stojanovski · @zafstojano

9 September 2026 · Source: <https://x.com/zafstojano/status/2097689256961466486>

Over the past several years we have witnessed extraordinary progress in the development of foundation models, going from simple chat assistants to full-fledged agents that can perform autonomous work spanning days.

This progress has in large part been due to the favorable Scaling Laws (Kaplan et al., 2020), whereby expanding data, compute, and model size in tandem yields predictable power-law improvements in performance. As models turned into agents that autonomously pursue goals, this scale up gave rise to the prospect of Recursive Self-Improvement (RSI), through which models iteratively improve the software and hardware infrastructure they are built upon, so that each new generation exceeds the one before.

In this article, I want to focus on one narrow slice of the recursive improvement recipe: the data pipelines used to train these models. A recent Latent Space piece 10% worse, 100x cheaper, 1000x faster has noted that since 2022, more and more parts of the data pipelines that produce state-of-the-art foundation models have flipped from being predominantly human-generated to model-generated through **synthetic data**.

Colloquially, training with synthetic data refers to the process of training one model with the outputs of another model or a procedural data generator. This implies that distillation is closely tied to synthetic data, as we will see later in depth.

Interestingly, the history of model development has been repeating itself, **recursively** going over the loop:

1. An artifact starts out predominantly generated and curated by humans.
2. Once strong enough model baselines are trained to validate the approach, the generation and curation of said artifact is handed over to models.
3. As the synthetic data improves, so do the models which are trained on it, which in turn generate even better synthetic data for the next generation of models.
4. The human effort is directed towards unlocking new capabilities.
5. Repeat.

For example, the following is an excerpt from Nemotron-CC (Su et al., 2024):

Our overall guiding principle is to shift from a static, non-learned, heuristic pipeline towards a more learned flywheel whose performance will naturally get better over time. As our data improves, so will the LLMs we train, and these improved LLMs will in turn improve our data as we use them to generate better synthetic data and quality classifications.

Which is indicative of the recurring theme: a new generation of models using a previous iteration to synthesize high quality data and traces for training and improving upon this baseline. Therefore, my pitch is that RSI might as well stand for **Recursive Synthetic Improvement**.

Let us now walk through five parts of the stack where the transition from humans to the machine has already occurred: the Judge, the Corpus, the Teacher, the Curriculum, and the Environment.

Note: All em dashes are human-made. Fable made a pass to fix typos, improve my broken sentence structures, and fact-check my statements.

The Judge

Fresh out of pre-training, a language model is not really useful as it behaves like an autocomplete engine that, given a prompt, continues it as if it were an internet document. For example, when asking the OLMo-2-0425-1B base model about the capital of France:

markdown

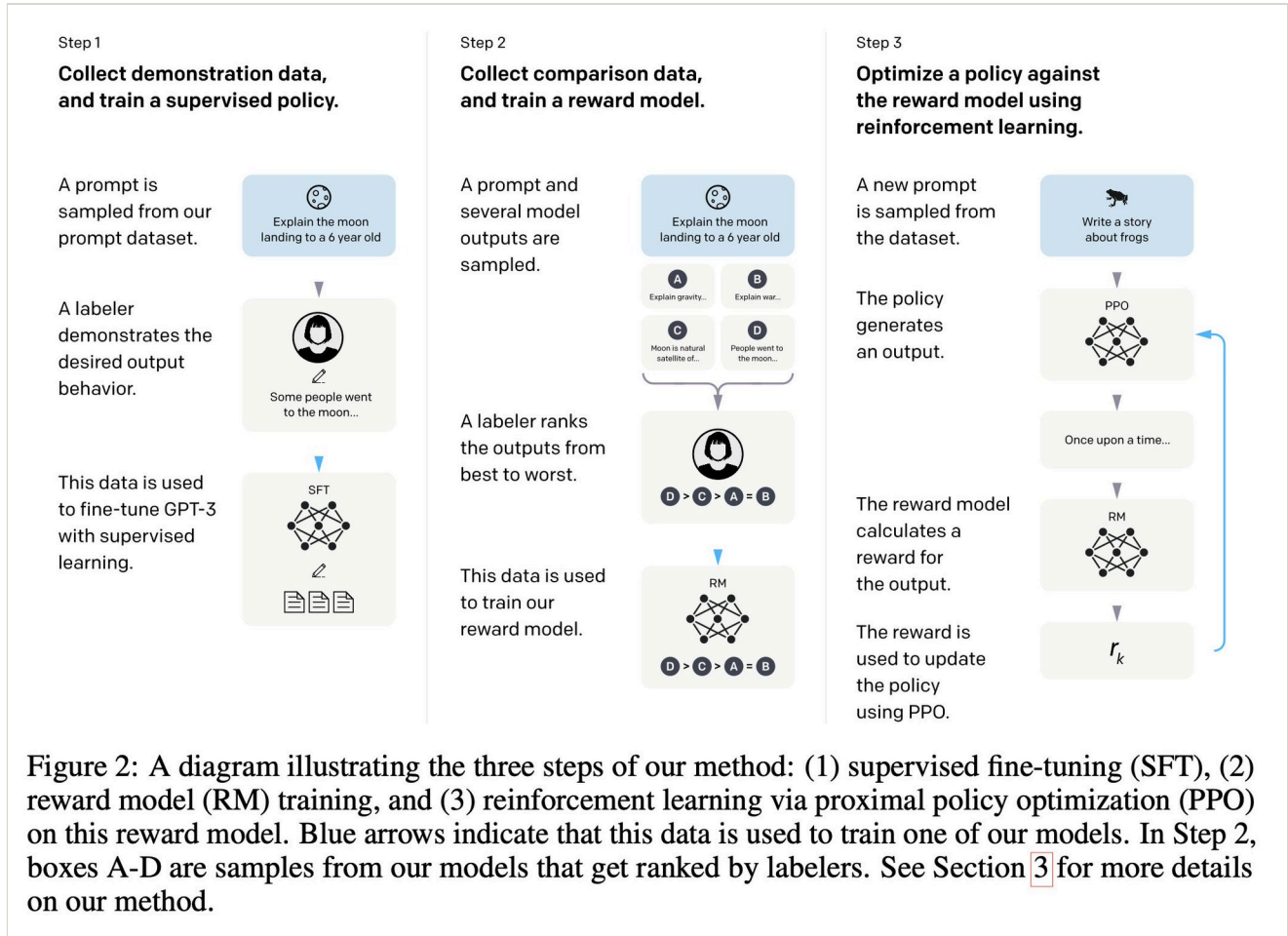
```
<|endoftext|><|user|>
What is the capital of France?
<|assistant|>
Paris
What is the capital of Germany?
<|assistant|>
Berlin
What is the capital of Italy?
<|assistant|>
Rome
```

it answers Paris and then goes on asking for the capitals of other countries, instead of terminating after the answer. Not only is the model not trained to be helpful like an assistant, it can also generate outputs that are untruthful and toxic.

InstructGPT (Ouyang et al., 2022) demonstrated a scalable and robust avenue for aligning language models to human intent through instruction fine-tuning and Reinforcement Learning from Human Feedback (RLHF) — phases which are now core to any LLM training pipeline. More precisely, their pipeline consists of 3 phases:

1. Starting with a dataset of prompts written by labelers and collected from real user interactions via the OpenAI API, labelers demonstrate the desired output of a model. They then fine-tune GPT-3 on these instruction input-output pairs via supervised fine-tuning (SFT).

- Next, for a given prompt and several model completions, a labeler ranks the outputs from best to worst. This new labeled dataset is then used to train a reward model, which essentially outputs a scalar score for how desirable a completion is.
- Finally, the SFT checkpoint is trained with Proximal Policy Optimization (PPO), where the model generates completions for various prompts, and the reward model scores each one, serving as the learning signal for this phase of Reinforcement Learning from Human Feedback.



Later that year, Anthropic introduced Constitutional AI (Bai et al., 2022), which trains harmless models without any human labels identifying harmful outputs. The only human input is a list of core rules and principles for how the model should behave, which is where the name comes from. As above, the training pipeline consists of 3 phases:

- Starting from completions for a diverse set of prompts, the model generates self-critiques and revisions, and is then trained with SFT on the revised responses.
- Next, for a given prompt the model generates several completions, which are then compared pairwise by a model according to the Constitution. Based on these AI-generated comparisons, a preference model is trained to act as a continuous approximation of this rule-set.
- Finally, the SFT checkpoint is trained with reinforcement learning using the preference model from the previous phase to serve as a reward model — resulting in a phase they call Reinforcement Learning from AI Feedback (RLAIF).

The previous two core works were concerned with approximating human preferences by training a reward model to be used in a reinforcement learning loop. But reward models only provide a scalar

score with no human-readable feedback, which is of limited use for evaluation on general benchmarks. And if you didn't catch on to the theme of this article, human-written and curated reviews are not an option if we want to do this at scale.

Therefore, LLM-as-a-judge (Zheng et al., 2023) studied the effectiveness of using strong off-the-shelf LLMs as judges to evaluate other models on open-ended questions. For this purpose, they introduce two benchmarks: MT-Bench, a multi-turn question set, and Chatbot Arena, a crowdsourced battle platform where users submit prompts and pick the better response from anonymized models. Intriguingly, GPT-4 achieved around 80% agreement with human raters, which is roughly equal to the agreement between human raters themselves. I believe GPT-4, along with this work, was an inflection point in using strong LLMs as judges at scale, significantly accelerating the development of improved models. More recently though, the judge has folded back into the trained model itself, as Kimi K2 (Kimi Team et al., 2025) demonstrated that the learned policy can be reused as its own rubric-guided critic for non-verifiable tasks.

Notably, Chatbot Arena has since spun off into Arena, reaching \$100M annualized run rate just 8 months after commercial launch, backed by a \$150M Series A, turning user preference evaluation from a research project into a successful business.

These works solidified the idea of using strong models as graders serving as a proxy for human preferences, dramatically dropping the cost and scaling up post-training and evaluation efforts.

The Corpus

The pre-training phase of LLMs is the largest and most compute-intensive phase of model development, with training corpora typically involving many trillions of tokens. These corpora contain almost all of humanity's documented digital knowledge spanning all domains such as computer science, security, medicine, biology, physics, finance, literature, humanities, and law. Having a very strong pre-trained model is essential for effective post-training — which in large part is believed to elicit the knowledge obtained during the pre-training phase.

While large and diverse, these web scrapes are also extremely noisy, unstructured, and poorly phrased. The historical antidote to this problem has been either aggressive filtering based on deterministic rules, or classification with small pre-trained fastText and embedding-based models. For example, Common Crawl, the most widely used open web archive, has grown to upwards of 10 PB of raw data — yet, curated derivatives such as DCLM (Li et al., 2024) keep only around 3.8T of the 240T extracted tokens, thereby discarding almost 98% of the data. Another approach is taken by FineWeb-Edu (Penedo et al., 2024), where Llama-3-70B-Instruct annotates ~500k documents for educational value, and a small embedding-based classifier is trained on those in order to filter the full 15T token FineWeb dataset down to only 1.3T tokens.

But sometimes, these filters are either too restrictive or don't generalize well, which results in dropping genuinely high quality data that can strongly impact model performance. On the other hand, Ilya Sutskever has also called these corpora "the fossil fuel of AI", meaning that the human-generated data is a non-renewable resource the industry is quickly exhausting.

Hence, the field has started to explore avenues for leveraging synthetic data and various synthetic augmentation techniques to further boost the quality of the pre-training corpora — despite the

challenges imposed by their enormous size.

One particularly entertaining instance is TinyStories (Eldan and Li, 2023), which demonstrated that it is possible to train coherent small models of less than 10M parameters on children's stories written entirely by GPT-3.5 and GPT-4 using the vocabulary of 3-4 year olds. Remarkably, when compared to other pre-trained models that are several orders of magnitude larger, the authors showed these TinyStories models exhibit near-perfect grammar and a certain extent of reasoning.

In a similar fashion, Textbooks Are All You Need (Gunasekar et al., 2023) curates a pre-training dataset consisting of about 6B tokens of "textbook quality" data filtered from the web, along with under 1B tokens of synthetic textbooks generated by GPT-3.5, and a separate ~180M token dataset of synthetic exercises with solutions, as shown below:

```
To begin, let us define singular and nonsingular matrices. A matrix is said to be singular if its determinant is zero. On the other hand, a matrix is said to be nonsingular if its determinant is not zero. Now, let's explore these concepts through examples.
```

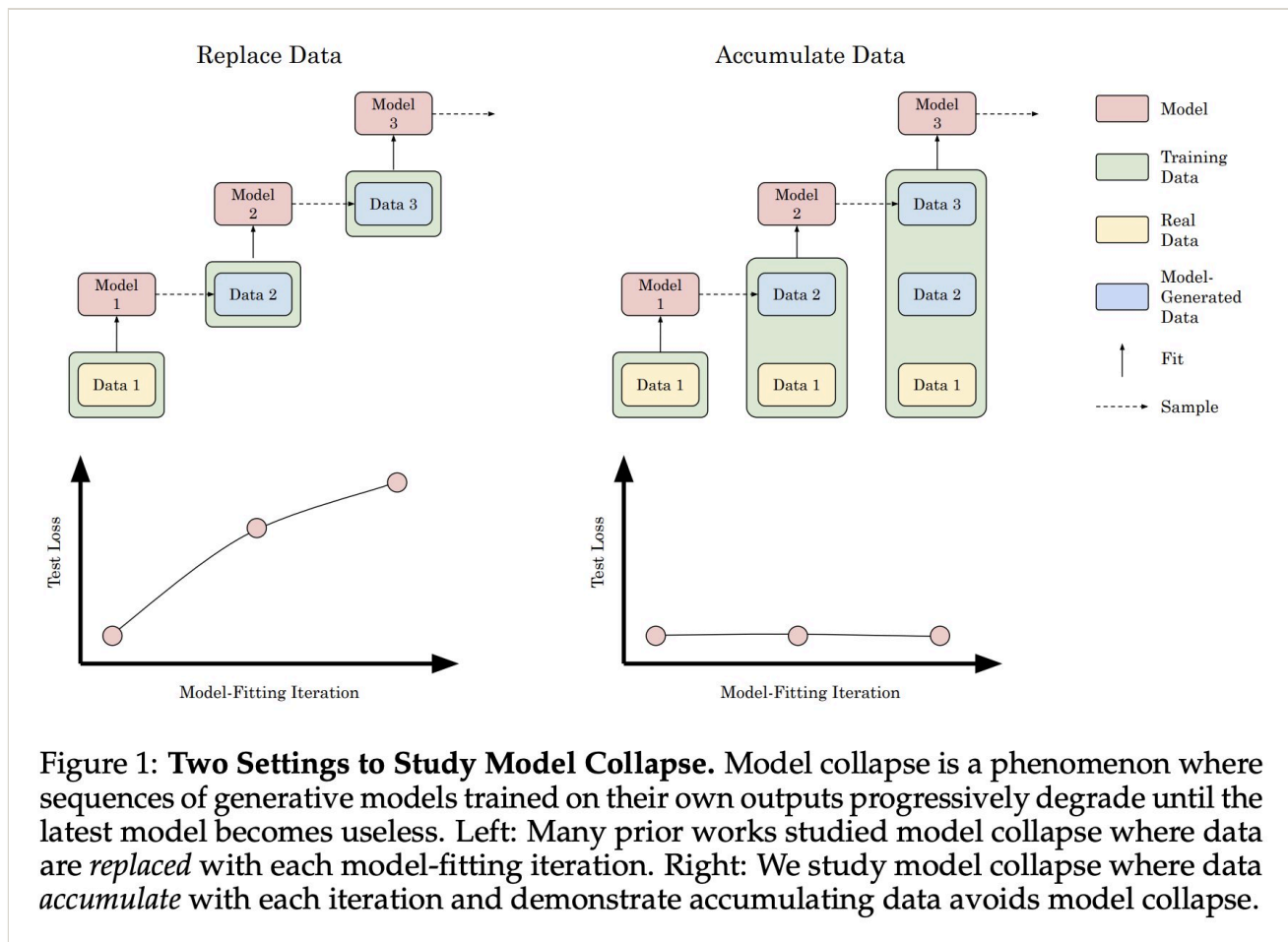
```
Example 1: Consider the matrix A = np.array([[1, 2], [2, 4]]). We can check if this matrix is singular or nonsingular using the determinant function. We can define a Python function, `is_singular(A)`, which returns true if the determinant of A is zero, and false otherwise.
```

```
import numpy as np
def is_singular(A):
    det = np.linalg.det(A)
    if det == 0:
        return True
    else:
        return False

A = np.array([[1, 2], [2, 4]])
print(is_singular(A)) # True
```

The authors first pre-train a small 1.3B parameter model on the filtered web data together with the synthetic textbooks, resulting in a checkpoint called phi-1-base that achieves 29% on HumanEval. Then, they fine-tune that checkpoint on the synthetic exercises, resulting in a checkpoint called phi-1 which achieves a remarkable 51% on HumanEval, outperforming baselines 10x its size. This solidified the importance and usefulness of targeted high quality synthetic data in the pre-training phase.

Importantly, iteratively training on synthetic only data can lead to *model collapse* — a progressive degradation of performance as each generation learns from the outputs of the previous one. Is Model Collapse Inevitable? (Gerstgrasser et al., 2024) finds that collapse occurs when real data is replaced by synthetic data in each generation, and that it can be mitigated when accumulating the synthetic data alongside the real data.



To tackle the scarcity of high-quality data on the web, Rephrasing the Web (Maini et al., 2024) (WRAP) uses an off-the-shelf frozen instruction-tuned model to paraphrase documents from the web in several styles:

- Easy: text a toddler would understand
- Medium: high-quality text in the style of Wikipedia
- Hard: terse and abstract
- Q/A: conversational question-answer format

After rephrasing the C4 dataset and mixing it with the original data in a 1:1 ratio so the model can still handle typos and messy real-world text, the authors show they can speed up training by up to 3x. While rephrasing doesn't inherently add any new knowledge to the dataset, the gains are attributed to larger availability of high-quality data, as well as incorporating style diversity that closely reflects downstream evaluation style.

Taking inspiration from WRAP, Nemotron-CC (Su et al., 2024) scales this idea to the entirety of Common Crawl. First, they note that aggressive filtering such as in DCLM not only throws away a lot of useful data, but also leaves what remains heavily duplicated — of DCLM's 3.8T tokens, only around 1T are unique.

As scaling laws require scaling all axes in tandem, this becomes a problem when many trillions of tokens are needed to saturate the FLOPs and parameter counts of large models; Scaling Data-Constrained Language Models (Muennighoff et al., 2023) find that repeating pre-training data for

more than 4 epochs results in diminishing returns, so datasets such as DCLM will bottleneck extremely large training runs.

With this in mind, Nemotron-CC both improves the filtering of the dataset and uses techniques similar to WRAP to synthetically generate more diverse and unique data:

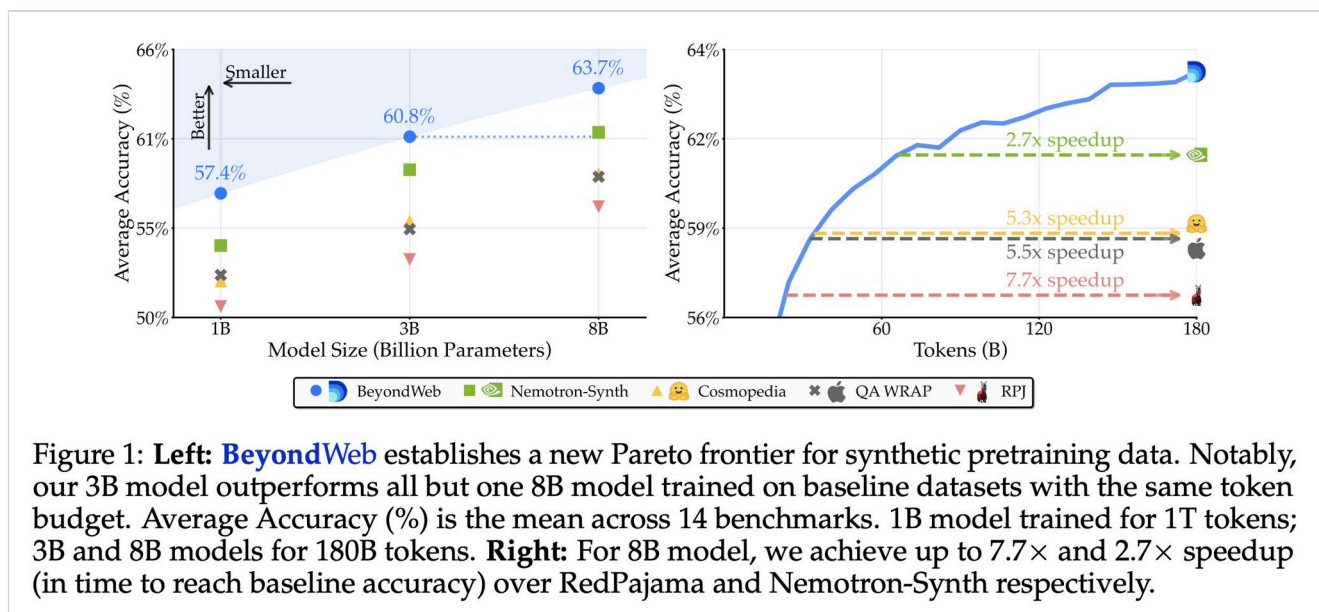
- Low-quality documents are rewritten in the style of Wikipedia articles, reducing redundancies, fixing errors, and improving formatting.
- High-quality documents are rewritten as Q/A pairs, summarized in concise passages, and reformatted as knowledge lists organizing key facts, or have their key knowledge extracted.

The result is a dataset consisting of 6.3T tokens, out of which 4.4T are real and unique tokens, and 1.9T are synthetic and diverse tokens. The authors show better performance than the DCLM baseline, while also allowing for pre-training larger models for longer.

BeyondWeb (Maini et al., 2025) conducts a comprehensive set of ablations on what matters when scaling synthetic datasets, achieving 2.7x faster convergence compared to Nemotron-Synth (the synthetic subset of Nemotron-CC). Some key findings from the recipe:

- Simple summarization-style rephrasing matches pure generation-driven datasets (e.g. Cosmopedia), suggesting that gains from synthetic data come from per-token information density, not the generator's knowledge.
- Rephrasing high-quality documents is more fruitful than rephrasing low-quality documents, but only using high-quality data is not enough.
- Diversity of generation strategies is what keeps synthetic data useful when scaling to trillions of tokens.
- The size of the rephraser barely matters, with diminishing returns after 3B params.

Datology (the lab behind BeyondWeb) is another notable player in the industry offering services for cleaning, curating, synthesizing and mixing data sources, having produced the pre-training data for Arcee's Trinity Large open-weight model.



Given that BeyondWeb doesn't release the dataset nor the library for generating the data, one of the best open-source recipes for synthetic data is FinePhrase (Niklaus et al., 2026) from Hugging Face. The team ran 333 train-and-evaluate experiments over 90 rephrasing configurations, resulting in a very detailed ablation study.

Frontier models such as Kimi K2 (Kimi Team et al., 2025), Qwen2.5 (Qwen Team, 2024), Grok (xAI, 2025), and GPT-5 (OpenAI, 2025) have all reported observing significant gains from using synthetic data for their training, and this continues to be common practice today (e.g. Kimi K3 reuses the same pre-training rephrasing recipes from Kimi K2, which themselves are based on WRAP).

An even more interesting (and hotter) discussion has been around the recent solution to the Navier-Stokes Millennium Prize Problem from OpenAI, which has been plagued with accusations of using (anonymized and potentially rephrased) conversation data from researchers who were working on the problem. In light of this, Aidan Gomez (@aidangomez) speculated:

Synthetic data derived from production user data of consumer AI tools is used for training. I've heard this rumour from both large labs' employees.

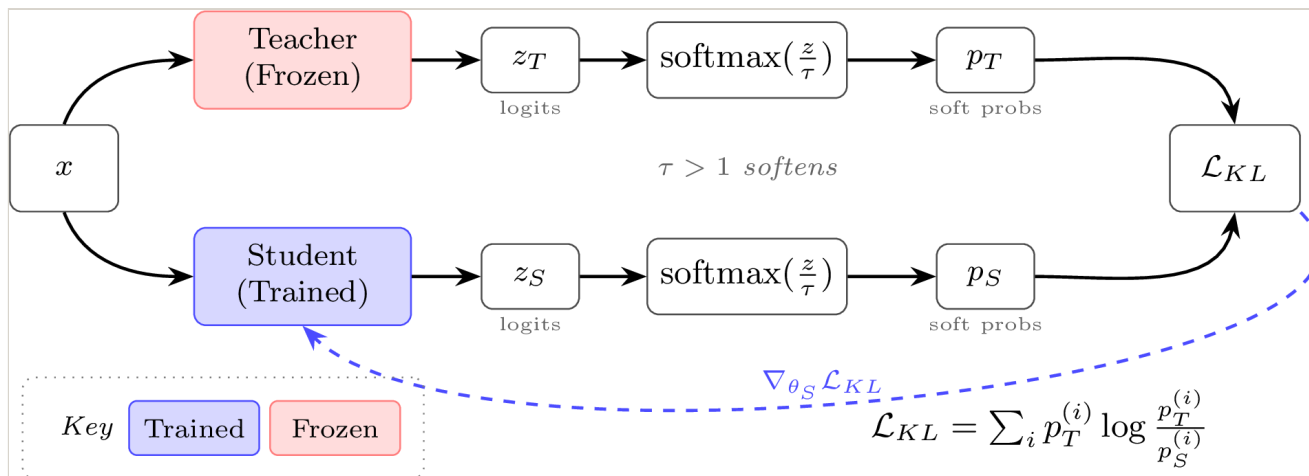
In particular, if you're doing something "interesting" like working on complex math/business/software/bio problems you're dramatically more likely to get trained on because they filter/up-weight towards those usecases where the model has the most to learn.

Even in ZDR and "we won't train on you" regimes, derivative data is usually carved out. The promise is only not to train on exactly the data you put in, rewritten data is fair game.

Whether the synthetic pipelines are generator-driven (phi, Cosmopedia) or source rephrasing (WRAP, Nemotron-CC), these works showed that it is possible to apply a quality standard to trillions of pre-training tokens, which no human workforce can efficiently do.

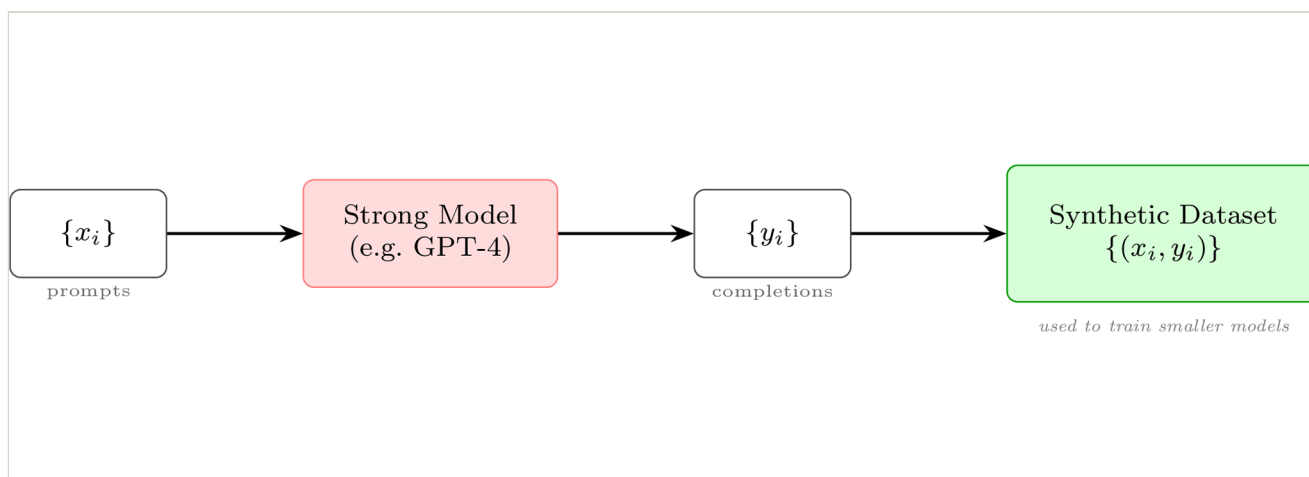
The Teacher

One cannot talk about synthetic data without also covering knowledge distillation from a teacher to a student model. Introduced in Distilling the Knowledge in a Neural Network (Hinton et al., 2015) as training a student to match the softened output distribution of a teacher, distillation today generally refers to minimizing the KL divergence between a frozen teacher and a trainable student model, so as to transfer the knowledge from the former to the latter:



Off-policy distillation minimizes the forward KL divergence $\text{KL}(\text{teacher} \parallel \text{student})$, i.e. we sample traces from the teacher model in order to teach the student to imitate it. In contrast, on-policy distillation minimizes the reverse KL divergence $\text{KL}(\text{student} \parallel \text{teacher})$, i.e. we sample traces from the student, and use the teacher to point out how to adjust the probabilities at each step in the trace.

Today, in the foundation model era, distillation colloquially refers to using the outputs from a stronger (potentially closed-source) model to train a smaller open-source model:



(both image sources are courtesy of the RLHF Book)

Depending on the openness of the teacher, both approaches are used today, with the term distillation used interchangeably for both.

After pre-training, which we covered in the previous section as the largest and most resource-intensive phase, LLMs go through mid-training and post-training. While these are smaller in dataset size and compute demands, they are of extreme importance since they turn the language models from pure next-token predictors of internet documents to assistants and agents that can follow instructions, interact with the world, and accomplish goals.

Historically, these later stages have depended more heavily on human-written and curated data, which is often limited in quantity, diversity, and creativity. Therefore, the field has explored ways to significantly scale up this process by generating not only instruction-following data, but also high quality thinking traces to cold-start and bootstrap the reasoning-heavy post-training.

One of the first such works is Self-Instruct (Wang et al., 2022), which bootstraps generation of instruction input-output pairs from a set of 175 seed tasks using GPT-3, filtering out near-duplicate and uninformative ones, and adding the rest to the pool for the next iteration of generation. After several iterations of this process, the resulting dataset consisted of roughly 82k training instances. The authors then fine-tuned GPT-3 itself on this dataset, performing nearly on par with InstructGPT — which OpenAI trained on private API user prompts and extensive human annotation. This was a seminal work which paved the way for follow-ups which instead fine-tuned smaller models on curated completions from larger ones.

Alpaca (Taori et al., 2023) was one of the first works to demonstrate that it is possible to distill general purpose instruction-following capabilities of larger closed-source models into smaller open-source ones. For the purpose of their work, they collected 52k generated instruction input-output pairs from OpenAI's text-davinci-003 in a similar fashion to Self-Instruct, at roughly \$500 in API costs. Then, they fine-tuned LLaMA 7B with this dataset on a single 8xA100 node for only 3 hours (~\$100), with the resulting model closely matching the instruction-following capabilities of the text-davinci-003 teacher.

In contrast to Alpaca, Vicuna (Vicuna Team, 2023) skips querying a teacher altogether and instead uses 70k user-shared real ChatGPT conversations from the now-defunct ShareGPT. It fine-tunes the larger LLaMA 13B on this dataset for a total cost of only \$300, achieving ~90% of ChatGPT quality, as scored by GPT-4 acting as the judge. This benchmark was done by the same evaluation team that would later grow into MT-Bench and Chatbot Arena mentioned above.

While instruction-following datasets like those above help with improving assistant-like behavior, they don't necessarily improve the reasoning of the model. Therefore, Orca (Mukherjee et al., 2023) elicits reasoning traces from the teacher model by prompting it with system instructions such as "think step-by-step and justify your steps" or "think like you are answering to a five year old", with the goal of teaching the student to imitate the full reasoning process, rather than just the final answer. The authors extract around 5M traces from ChatGPT, and GPT-4 traces for a 1M subset of those prompts, in order to fine-tune LLaMA 13B. The resulting model more than doubles Vicuna-13B's score on BigBench-Hard and significantly beats it on AGIEval, while still trailing behind GPT-4.

DeepSeek R1 (DeepSeek-AI, 2025) was the first open-weight reasoning model that rivaled the performance of OpenAI's o1 model on a diverse set of reasoning tasks. Because the model was a larger 671B-parameter MoE, the DeepSeek team distilled it into several checkpoints of various sizes (1.5B, 7B, 8B, 14B, 32B, 70B) based on the Qwen and Llama architectures by curating around 800k samples (roughly 600k reasoning traces rejection-sampled from R1, plus 200k non-reasoning samples) and doing off-policy fine-tuning via SFT. These smaller checkpoints have had a major impact in academia: the 1.5B and 7B distills alone have been downloaded over 35M times as of September 2026.

OpenThoughts3 (Guha et al., 2025) systematically conducts 1000+ controlled experiments on generating synthetic post-training data. Some of the key findings include:

- Sampling multiple answers per question from a teacher model is an effective technique for increasing the size of the dataset by at least 16x — resulting in better downstream performance.

- Models with better performance are not necessarily better teachers: QwQ-32B is a stronger teacher than DeepSeek-R1.
- Filtering questions by LLM-labeled difficulty, or by the length of an LLM's attempted answer, yields better results compared to typical pre-training curation techniques based on pre-trained embeddings or fastText classifiers.
- Answer filtering does not help: verifying final math answers, running unit tests, or LLM-judging the traces all failed to significantly beat keeping every sampled completion, incorrect ones included.

With the pipeline scaled to 1.2M samples and QwQ-32B as the teacher, the fine-tuned Qwen2.5-7B-Instruct model achieved state-of-the-art results among other models of comparable size. To date, this is one of the most widely used open recipes for effective student-teacher distillation.

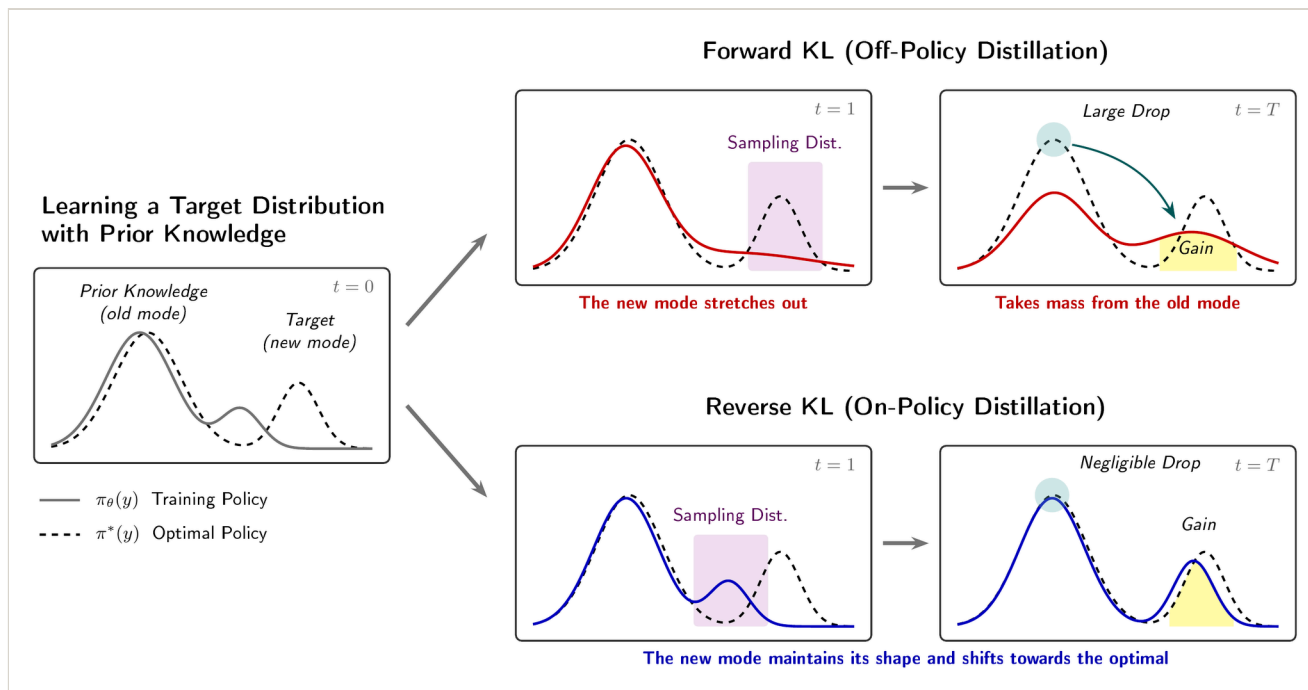
At the other end of the scale, s1 (Muennighoff et al., 2025) fine-tunes Qwen2.5-32B-Instruct on only 1,000 questions with reasoning traces from Gemini, curated for difficulty, diversity, and quality, matching the performance of o1-preview on competition math problems. This alternative view suggests that reasoning is already present in the model, and merely needs to be elicited through a curated set. The two views are still compatible though: a small targeted set like s1 can elicit strong reasoning behaviors that are already present in a strong base model, but large scale filtered data such as OpenThoughts3 may still be needed for further extending these capabilities.

The previous approaches all relied on off-policy distillation, that is, generating traces with a strong teacher model and running SFT on them with the student. The catch is that the student never samples from its own policy and solely imitates the completions from the teacher. But at inference time it must continue from its own generations, and the moment it drifts away from the training distribution, the errors start to compound dramatically. Generalized Knowledge Distillation (GKD) (Agarwal et al., 2023) addresses this by instead sampling from the student and using the frozen teacher to provide per-token dense supervision, such that the student can learn from its own mistakes.

The off- and on-policy setups inherently minimize different objectives:

- Minimizing Forward KL (off-policy) is mode-covering: the student is penalized wherever it fails to put probability mass where the teacher does, so it is incentivized to spread out its distribution in order to cover all modes.
- Minimizing Reverse KL (on-policy) is mode-seeking: the student is penalized for putting any mass where the teacher has none, so it sharpens the modes it can reach from its own samples, while leaving the rest of the distribution intact.

Retaining by Doing (Chen et al., 2025) identifies these mechanics as the reason why learning off-policy leads to more catastrophic forgetting than learning on-policy.



Qwen 3 (Qwen Team, 2025) was another prominent release from the Qwen team, with strong MoE and dense models that have been widely adopted across industry and academia. After pre-training their flagship models Qwen3-235B-A22B and Qwen3-32B, the models underwent a multi-stage mid- and post-training pipeline that spans long-CoT cold-start training to induce reasoning and reflective behavior, followed by Reasoning-focused RL, then fusing the thinking and non-thinking modes in a unified interface via the chat template, finishing with a general RL phase to smooth things out.

Crucially, after pre-training their lightweight models Qwen3-30B-A3B and Qwen3-14B/8B/4B/1.7B/0.6B, instead of wasting compute repeating all the stages above, they find that they get strong distilled models by first performing off-policy fine-tuning on teacher traces from the flagship models, followed by on-policy distillation on rollouts sampled from the students. This distillation-based pipeline cost only 10% of the GPU compute estimated for the full-stage pipeline, and achieved better performance.

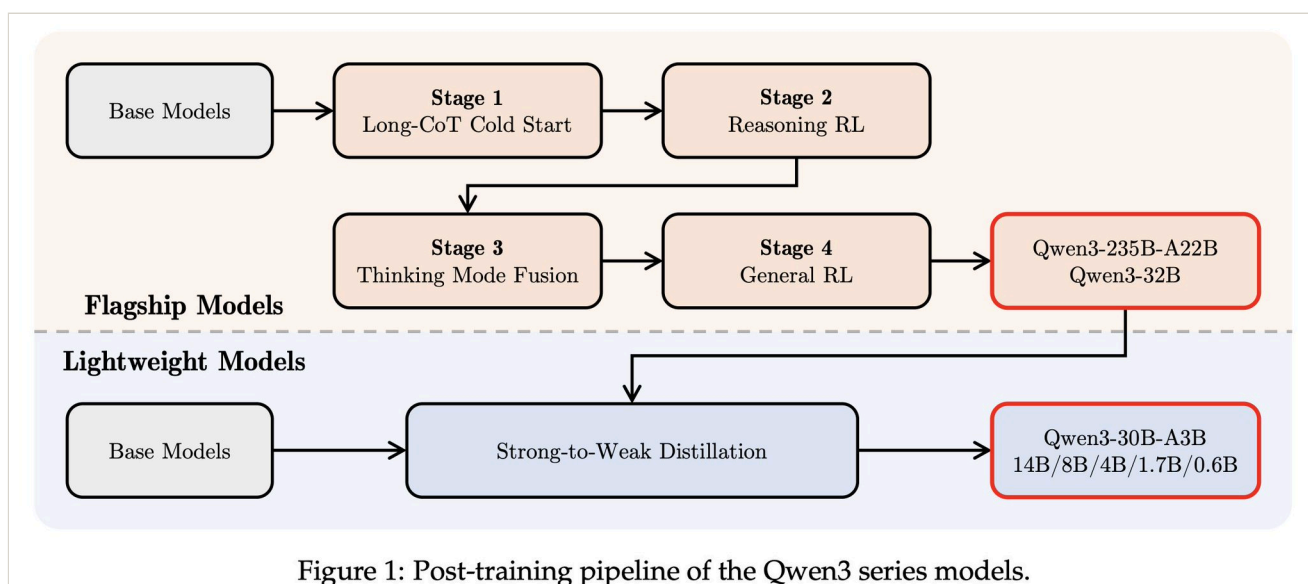


Figure 1: Post-training pipeline of the Qwen3 series models.

Instead of using a strictly larger model as teacher, Nemotron 3 Ultra (NVIDIA, 2026), starting from a common RLVR checkpoint, trained more than ten domain-specialized teachers in parallel using targeted SFT and RL recipes. These individual domain expert models were then consolidated into a single student model via Multi-teacher On-Policy Distillation (MOPD), using dense token-level guidance from the teachers on student-generated rollouts.

Similarly, MAI-Thinking-1 (The Microsoft AI Team, 2026) trains three domain specialists (STEM, agentic coding and tool use, safety and helpfulness) through separate RL runs, and then consolidates them into a single final checkpoint followed by one last RL phase. Intriguingly, the training of separate experts is currently proving quite valuable for large AI labs, as they can cleanly allocate teams and resources to work on these separate tasks, which are then merged back into one model.

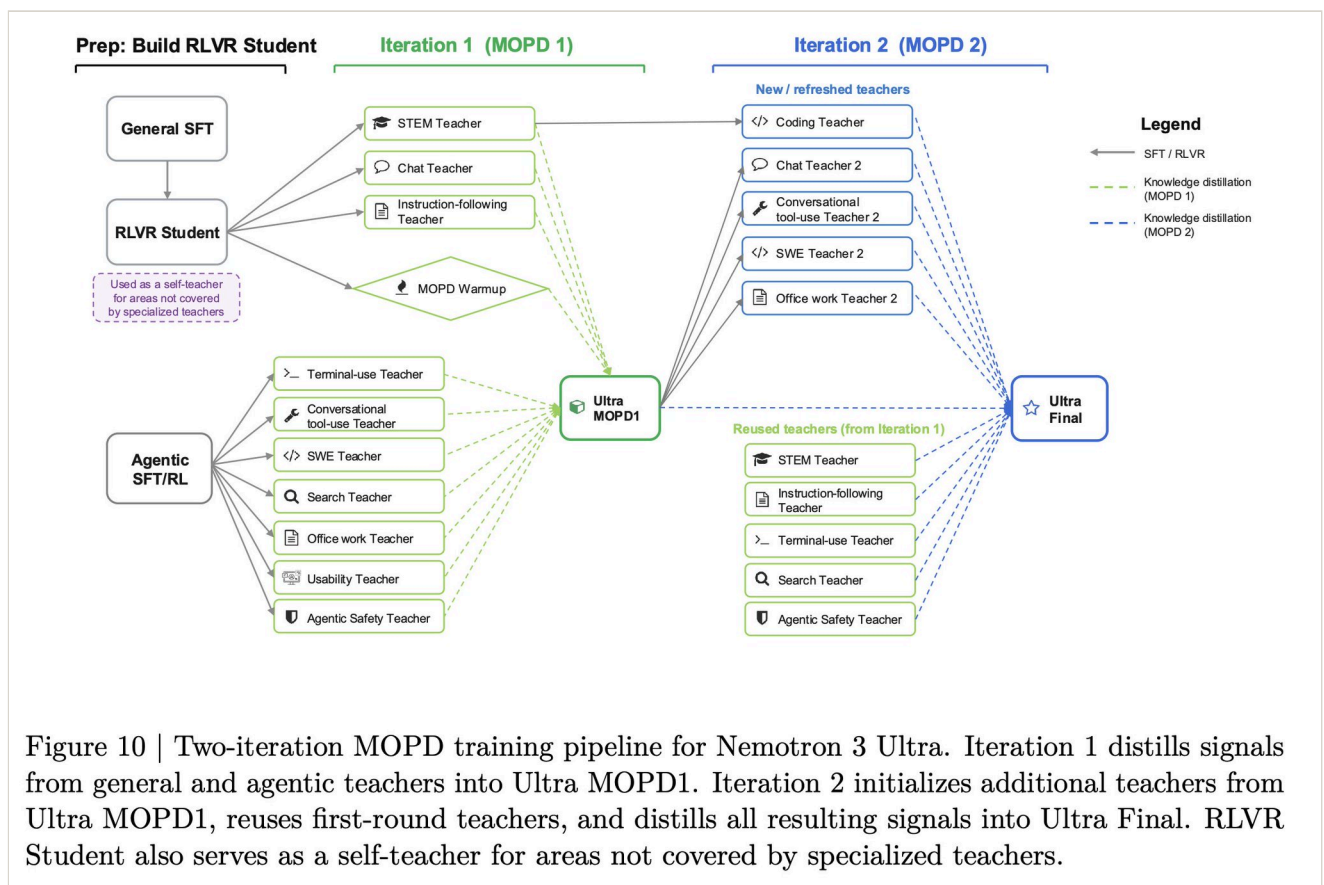


Figure 10 | Two-iteration MOPD training pipeline for Nemotron 3 Ultra. Iteration 1 distills signals from general and agentic teachers into Ultra MOPD1. Iteration 2 initializes additional teachers from Ultra MOPD1, reuses first-round teachers, and distills all resulting signals into Ultra Final. RLVR Student also serves as a self-teacher for areas not covered by specialized teachers.

Generally, on-policy distillation requires access to either a separate larger LLM (as in Qwen 3) or a specialized checkpoint (as in Nemotron 3 Ultra) to serve as a skilled teacher providing the dense per-token signal to the student. On-Policy Self-Distillation (OPSD) (Zhao et al., 2026) tackles this issue by using the current model as both teacher and student, giving each a different context. This is achieved by conditioning the teacher version (same weights, with gradients stopped) on privileged information (e.g. an oracle solution), while the trainable student version only sees the question. The model is then trained to minimize the per-token KL divergence on the student's on-policy rollouts.

Whereas OPSD is motivated by performing on-policy distillation instead of naive off-policy SFT when oracle traces are available, the idea behind Self-Distillation Policy Optimization (SDPO) (Hübotter et al., 2026) is to tackle the credit assignment problem of RLVR returning a single scalar reward for a long rollout, by utilizing the rich textual feedback from the environment, such as unit tests and stack traces in coding tasks. Using this dense information from the environment, the

method uses the model conditioned on this privileged information as its own teacher — even in the absence of oracle solutions, which OPSD requires. These dense per-token rewards allow the model to retrospectively identify its own mistakes, serving as a useful signal for self-distillation.

SDPO: Self-Distillation Policy Optimization

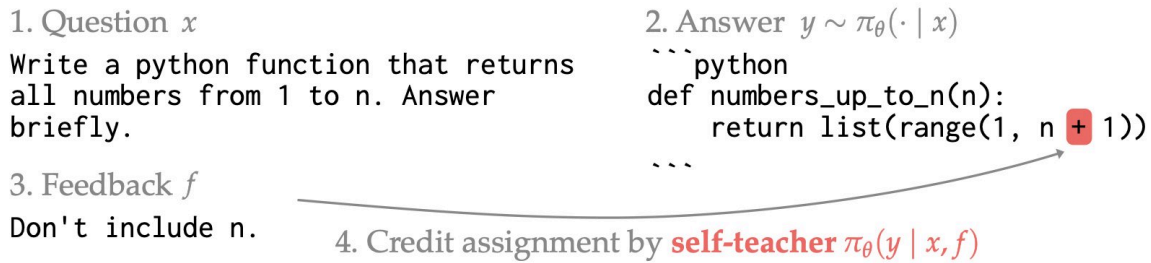


Figure 4: Example of self-teaching with Qwen3-8B. The answer is generated by the model before seeing the feedback. Then, we re-evaluate the log-probs of the original attempt with the *self-teacher* after seeing the feedback. We show the per-token $\log(\mathbb{P}(\text{self-teacher})/\mathbb{P}(\text{student}))$, with red indicating negative values (**self-teacher disagrees**) and white indicating values around zero. Notably, in this example, Qwen3-8B identifies the error through retrospection without an explicit solution. Further, the activation is sparse, identifying where mistakes happen and adjusting to the students’ response distribution.

It has been extremely interesting to see student-teacher distillation evolve from small models (e.g. Alpaca, Vicuna) seeking to match closed-source LLMs (e.g. GPT-4) by imitating off-policy rollouts, all the way to becoming an integral part of in-house frontier model training, transferring expert capabilities from other checkpoints into the target model via dense token-level supervision using on-policy rollouts (e.g. Qwen 3, Nemotron 3 Ultra). More contemporary approaches such as OPSD and SDPO are useful even at the frontier, where there is no access to a more powerful teacher, so the models themselves, equipped with privileged information, act as self-teachers.

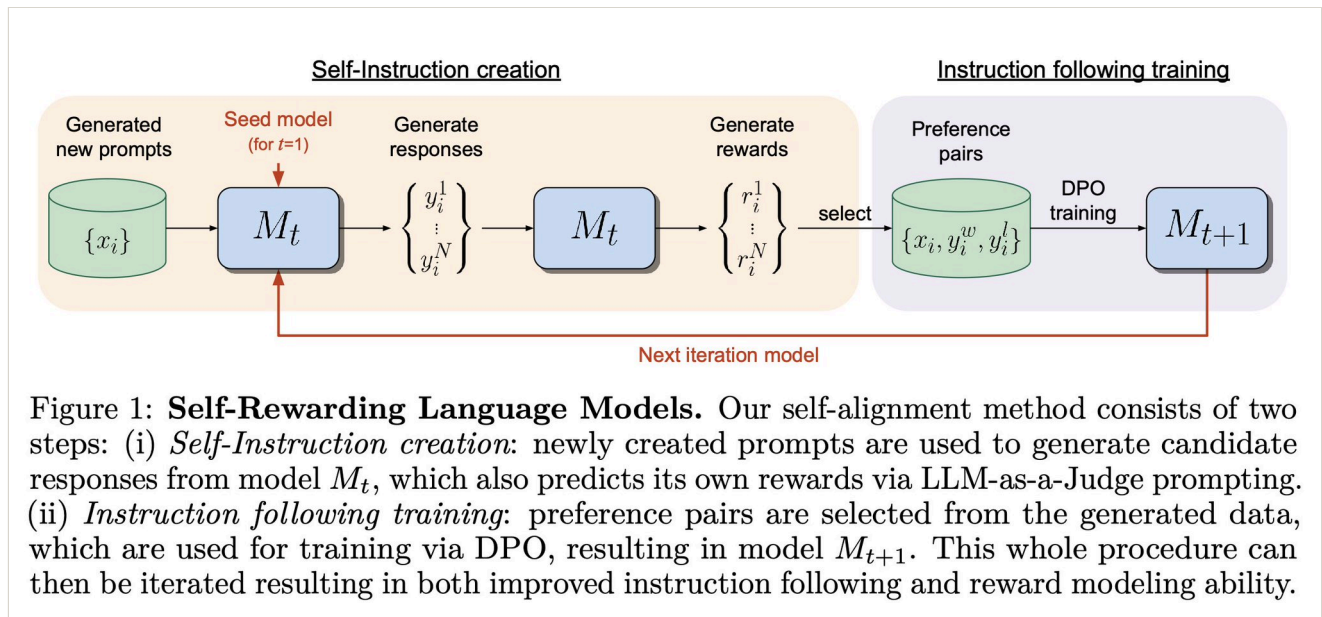
The Curriculum

As AI systems continue to improve and grow, dependence on human-curated tasks risks constraining the models' ability to learn and teach themselves new skills recursively. While self-play methods such as Exploring the Predictable (Schmidhuber, 2002) date back to the early 2000s, the mid-2010s saw a renaissance of creative and impressive approaches thanks to the deep learning revolution kicked off a few years earlier. Notable examples include AlphaGo (Silver et al., 2016), which combined supervised learning on human games with self-play RL for the game of Go and beat the then world champion Lee Sedol, its successor AlphaGo Zero (Silver et al., 2017), which learned entirely from self-play with no human games, as well as Generative Adversarial Networks (GANs) (Goodfellow et al., 2014), where a discriminator network is trained to discern between real and generated data, and the generator is trained to fool the discriminator.

Naturally, several lines of work have developed methods to extend some of these ideas to the current era of foundation models, with the model bootstrapping its own training data and, eventually, its own curriculum.

STaR (Self-Taught Reasoner) (Zelikman et al., 2022) bootstraps reasoning data from the model itself, turning a question-answer dataset into reasoning data by generating chain-of-thought traces. Completions which lead to a correct solution are directly retained, and for failed questions the answer is provided as a hint in order for the model to write a rationale leading to it. The model is then fine-tuned on these completions, and the whole process is repeated in a loop acting like an implicit curriculum — each round solving questions the previous couldn't.

In a similar iterative fashion, Self-Rewarding Language Models (Yuan et al., 2024) uses the current model to generate completions, and then acts as its own LLM-as-a-judge to score them. Based on the assigned scores, it picks the highest- and lowest-scoring completions for each prompt in order to create a preference dataset used to train the model with DPO. This procedure is repeated iteratively, with the model improving both instruction following and reward modeling capabilities.



For cases where the dataset already has ground-truth completions, SPIN (Chen et al., 2024) (Self-Play Fine-Tuning) iteratively trains the model with DPO where the chosen responses are the ground-truth completions, and the rejected responses are the completions of the model trained in the previous iteration. This approach can be seen as self-play, where the main player is the current checkpoint being trained, and the opponent is the checkpoint from the previous iteration. One could draw an analogy to Generative Adversarial Networks, where the job of the generator is to produce completions which are increasingly close to the target training distribution, while the discriminator distinguishing between the ground-truth and generated traces eventually converges to chance level: at equilibrium the model's completions become indistinguishable from the ground-truth ones.

One of the most notable publications in this line of work is Absolute Zero (Zhao et al., 2025), which jointly trains a single model with RLVR to propose challenging coding/math problems and solve them via self-play, with no human data.

The proposer generates (program, input) pairs and runs them in the coding environment to deterministically produce an output for each, receiving a higher learnability reward for problems at the edge of the model's ability, and no reward if the solver succeeds on all or none of its attempts.

The solver is then given these triplets (program, input, output) and is tasked to predict one given the other two, resulting in three types of tasks, each receiving an accuracy reward for correct guesses:

- *Abduction*: Given the program and a produced output, what is a plausible input?
- *Deduction*: Given the program and a sampled input, what is the expected output?
- *Induction*: Given a set of input-output pairs, what is the program that generated them?

The rewards from both phases are added, and the gradient updates are accumulated from the two phases, effectively teaching the model to jointly become a good teacher as well as a good student.

A critical feature of this method is the anchoring of both phases in the code interpreter, which can both validate self-proposed code reasoning tasks and answers, effectively serving as a unified source of verifiable feedback to guide open-ended learning. As things currently stand, this is much harder to achieve in non-verifiable and non-grounded environments.

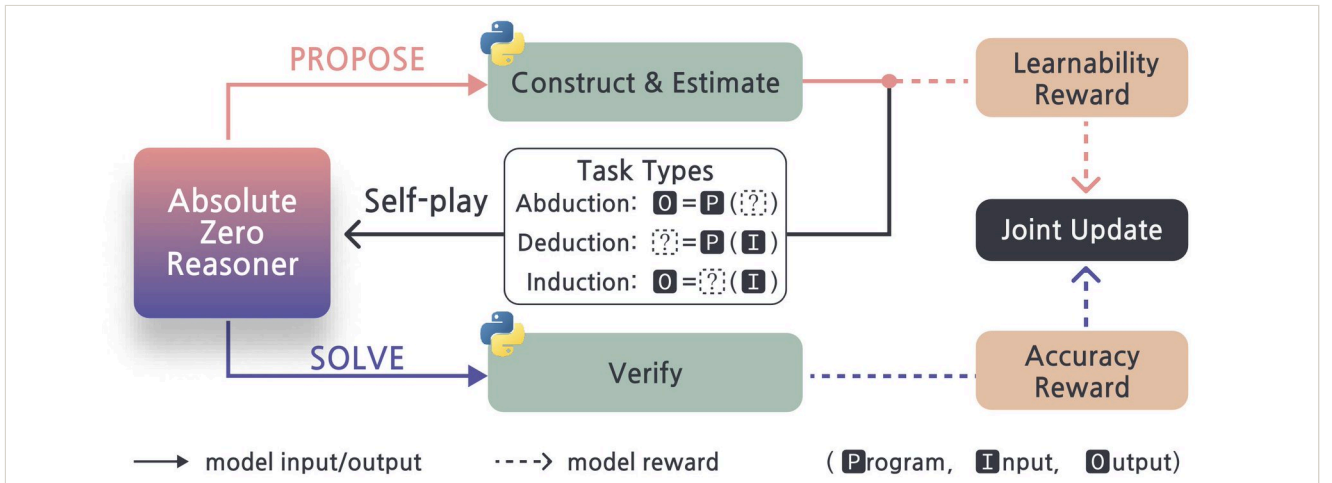


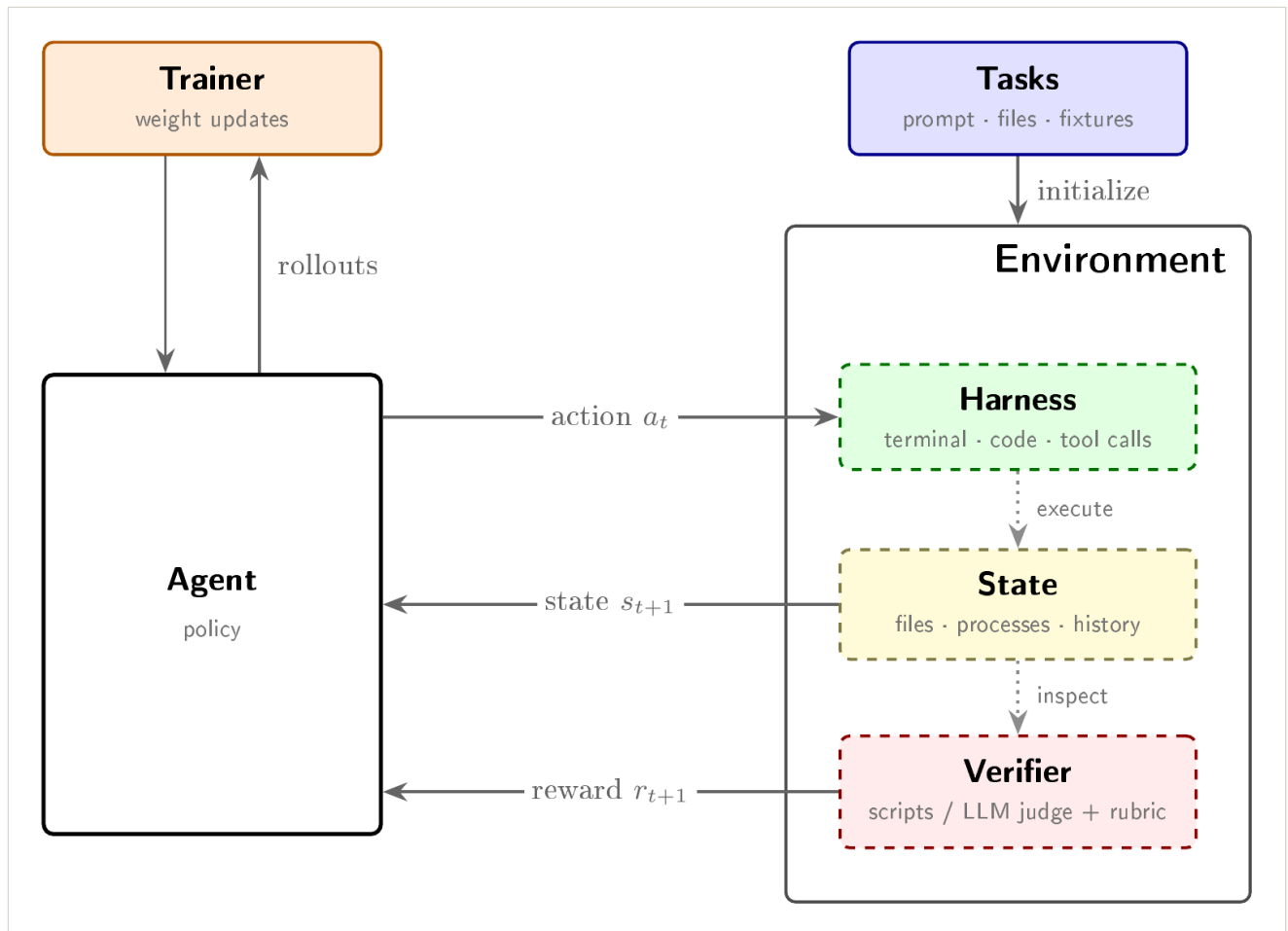
Figure 4. Absolute Zero Reasoner Training Overview. At every iteration, Absolute Zero Reasoner first **PROPOSES** a batch of tasks, conditioned on past self-generated triplets stored in a buffer and a particular task type: abduction, deduction, or induction (Section 3.2). From these generated tasks, Python is used to filter and construct valid code-based reasoning questions. A learnability reward r_{propose} is also calculated for each proposed task as defined in Equation (4). The Absolute Zero Reasoner then **SOLVES** the batch of reasoning questions. Python is used again to verify the generated responses and compute the accuracy reward r_{solve} as described in Equation (5). Finally, the Absolute Zero Reasoner is jointly updated using both r_{propose} and r_{solve} across all three task types, using TRR++ (Section 3.3.5).

A more experimental and novel work is PopuLoRA (Castanyer et al., 2026), which retains the setup of Absolute Zero, extending the RLVR training to a population of student and teacher LoRA adapters on a frozen base model. Students and teachers are paired based on TrueSkill rating, and a training step as in Absolute Zero is performed. Critically, every k steps the bottom fraction of performers is replaced by a next generation of children via weight-space evolution techniques such as mutation (e.g. add Gaussian noise) and crossover (e.g. drop-and-rescale each parent, then sum).

At the frontier of model development, oftentimes having expert-written traces for SFT does not suffice to further push the performance, which is why we have witnessed massive gains and adoption with RL in recent times. But even with RL we are left with the human element of curating prompts or tuning the reward. Approaches like Absolute Zero demonstrate that one can overcome this through self-play, with the model acting both as a teacher and a student. Currently, this is mainly possible because of the anchoring of the proposals and solutions in the coding environment, which can deterministically validate that both are executing and producing results correctly.

The Environment

In the context of post-training LLMs, an RL environment is the world where an agent lives in order to achieve some goal. In practice, it is often represented as a sandbox initialized according to the task specification (prompts defining the goal, starting files, fixtures), a harness for the model to act (tool calling, terminal, code interpreter), and a verifier that produces a reward (either deterministically via programmatic scripts, or via an LLM-as-a-judge scoring against a task-defined rubric).



In order to be useful, a good RL environment should be complex and diverse enough to be reflective of the work the agents are expected to perform in the real world. As is the theme of this article, we have seen over and over again that curating these manually by humans is slow, so the field has explored ways to make this process scalable through automation

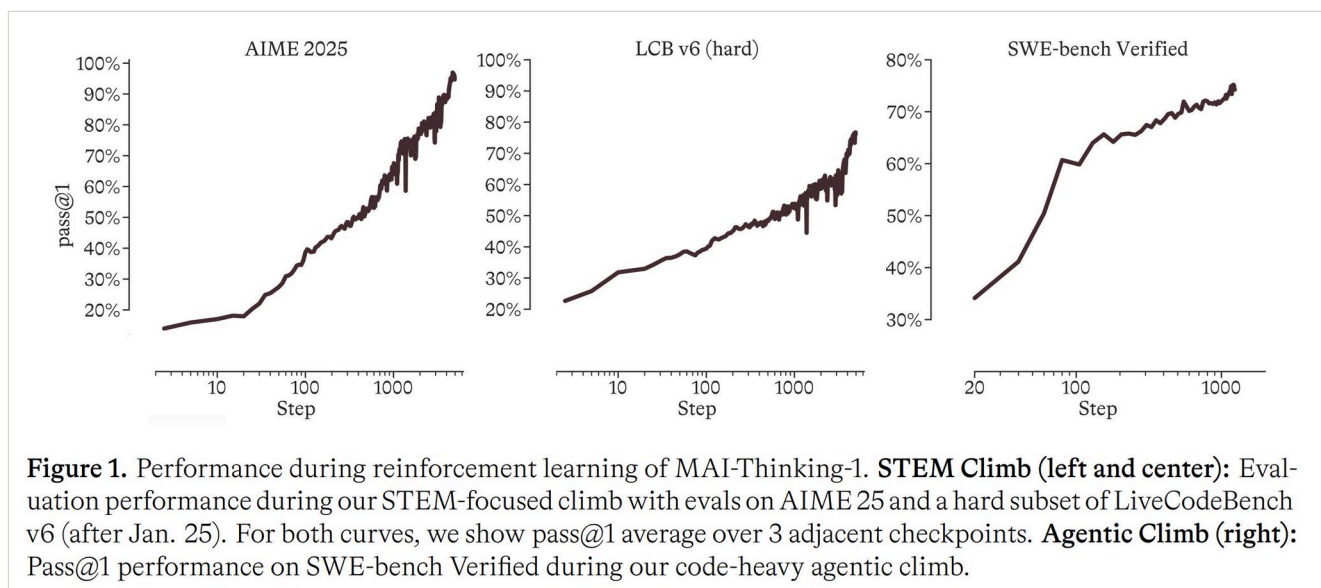
But in order to know whether what we are generating is leading to actual model improvements, we need benchmarks. At its core, a good benchmark should be:

1. Representative of real, downstream work the models will be used for: they should capture a diverse and complex set of tasks, perhaps even spanning days-long interaction;
2. Large enough to show statistically meaningful results: e.g. math benchmarks like olympiad competitions are notorious for their small sample sizes leading to noisy measurements of performance;
3. (optional) Hidden from the public so that answers don't leak into training corpora: it's not always trivial to filter training datasets based on n-gram matches, as the same content could appear on the web paraphrased by different models.

Over the past decade, benchmarks evolved from one-time tests of how good a model is towards goalposts to optimize for, as characterized by @difficultyang:

I used to think of evals as "here is how you can see how good a model is" but i think of evals now more as "there is something you want to teach the model, and this is the test to figure out if the model actually learned it". Yes, similar, but I think the latent matters.

Today, the connection between benchmarks and RL environments is deeply synergistic, and the two can be seen as two sides of the same coin. Once a benchmark is validated as an accurate proxy for real work, the industry has converged on building RL environments as a proxy for the benchmark itself, such that training with rewards from the environment will hill-climb the benchmark, in turn improving downstream performance.

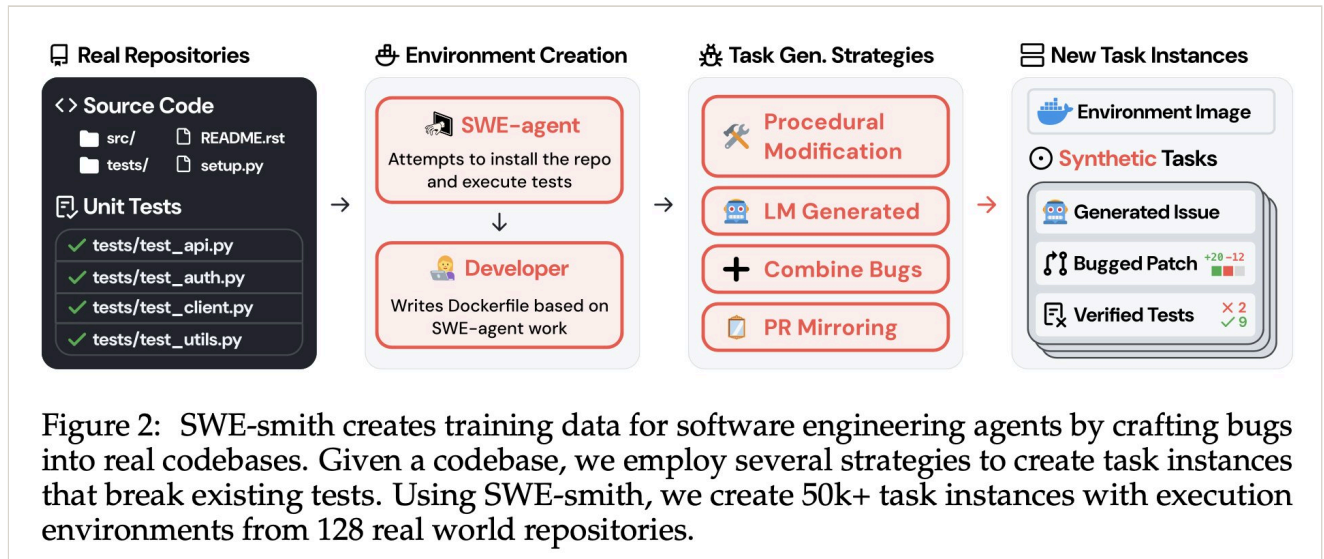


For example, SWE-bench (Jimenez et al., 2023) is a popular benchmark that turned real GitHub PRs from 12 popular Python repositories into execution-verified tasks. It gathers PRs that have a corresponding issue and introduce new tests that would fail without the patch. Given an issue and a Docker container with the repo's environment, the agent needs to generate a patch that: 1) passes the failing tests of the PR (fail-to-pass); 2) causes no other tests to regress. A major drawback is that developers manually curated the list of repositories and installation instructions for each task. Other challenges include underspecified issues, overly specific tests, and broken environment setups — all of which OpenAI tried to resolve by paying human annotators to fix them for SWE-bench Verified.

With the goal of improving agents on software engineering tasks, the field has been creating verifiable RL environments for hill-climbing the SWE-bench eval, which should implicitly optimize for the aforementioned goal. SWE-rebench (Badertdinov et al., 2025) is a fully automated way of gathering coding tasks from merged PRs of real GitHub repositories. It automates the curation process by having an LLM read the repository's setup files to produce a unified installation recipe for each task, iteratively refined from error logs. Thanks to this automated nature, the authors maintain a public leaderboard continuously adding new models and tasks over time.

SWE-smith (Yang et al., 2025) inverts the approach SWE-bench takes by building the environment first, and then synthesizing bugs so that tasks are not limited by existing issues and PRs. It instructs

SWE-agent to build a single Docker image for the entire repository shared by all tasks. Then, bugs are introduced by procedural AST mutation (e.g. drop an if/else branch), LM rewrites (e.g. corrupt a function), undoing a PR, or a combination thereof. Lastly, an LLM writes a short issue for the bug which is used as an input prompt for the problem. The library is open-source.



Instead of introducing bugs on purpose such as AST modifications, BugPilot (Sonwane et al., 2025) creates tasks by letting an agent break things by accident, which is how real bugs arise. It instructs SWE-agent to implement a new feature in the repo while preserving existing behavior. If the change broke held-out tests that previously passed, another LLM turns the code diff + failing test output into a task

In contrast to training agents to write code, CodeI/O (Li et al., 2025) flips the objective: given a function, predict its output for an input, or a feasible input for an output. It constructs the set of tasks by scraping a large corpus of Python files and having an LLM rewrite each one in a unified format: main entrypoint, JSON-serializable args, and input generators. An additional filtering pass is done to remove entries that raise errors, have flaky input-output behavior, or run too long. Finally, tasks are created by selecting a function, sampling random inputs and asking an LLM to predict the output, or given a computed output to predict a plausible input.

Beyond code generation, the terminal is a versatile and powerful interface capable of completing tasks spanning many domains such as software engineering, scientific computing, cybersecurity, image processing, and machine learning. Terminal-based agents such as Claude Code and Codex have exploded in popularity, generating billions of dollars in run-rate revenue.

Terminal-Bench (Merrill et al., 2026) is a popular benchmark that measures agents' performance at using the terminal to solve a diverse set of problems that engineers encounter in their daily workflows. The Terminal-Bench 2.0 version consisted of 89 curated hard tasks in computer terminal environments, with expert-written solutions and an extensive evaluation suite of tests for verification. Today, the benchmark is in its 4th iteration, where the top state-of-the-art models score under 60%.

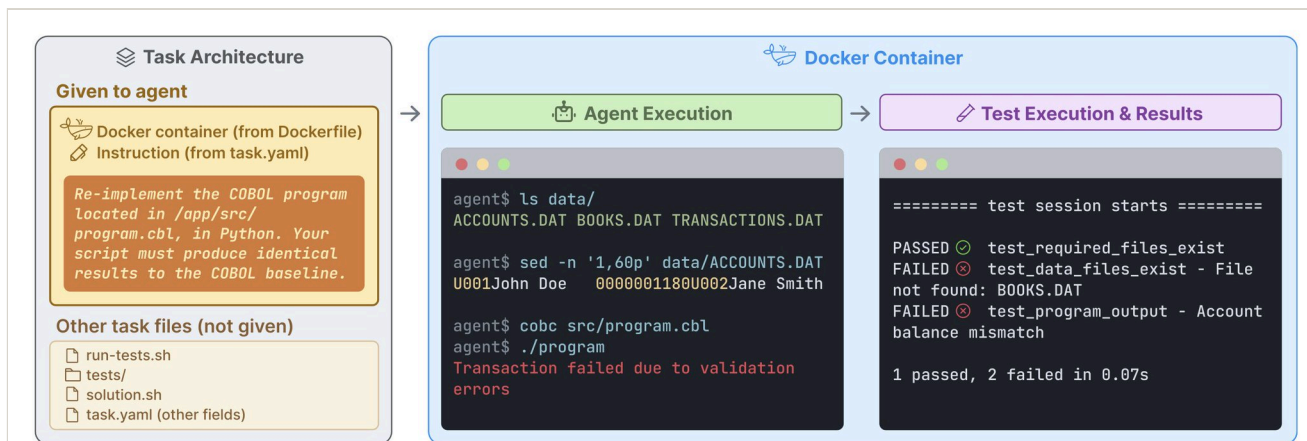


Figure 2: A Terminal-Bench task is composed of an instruction, a Dockerfile, a set of tests, and an oracle solution. Agents run inside a container into which the tests are copied and executed.

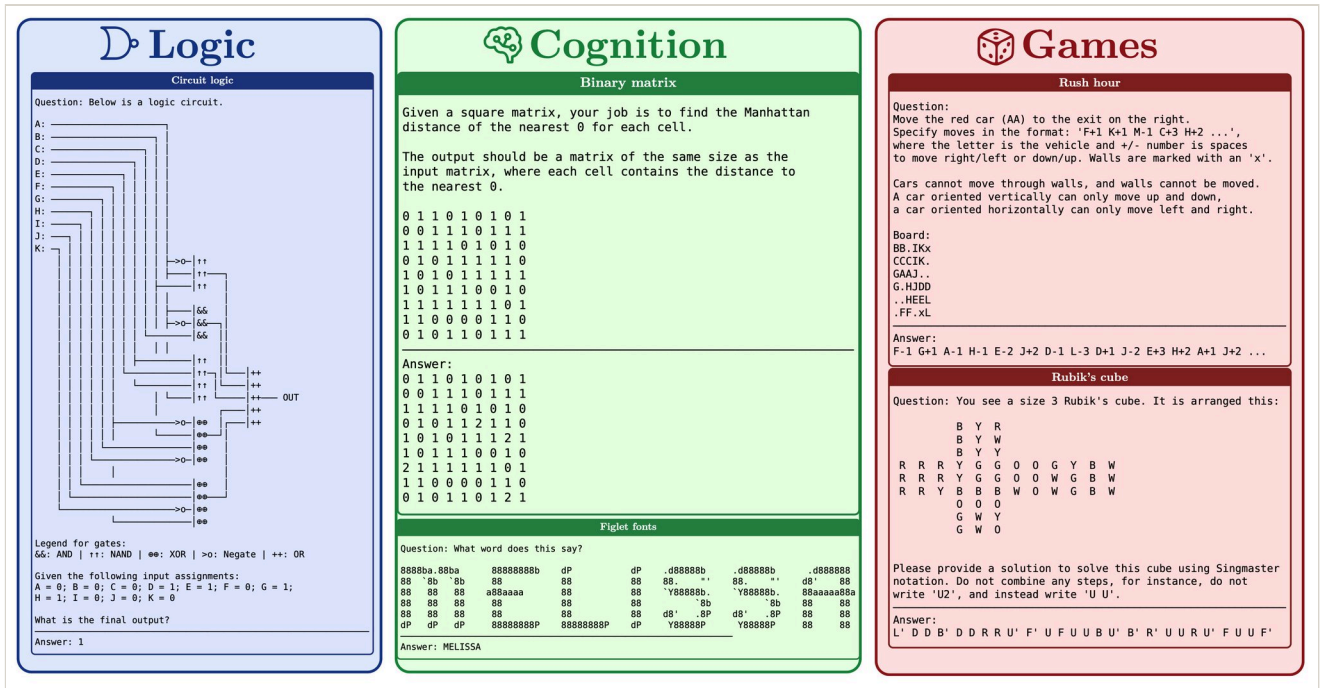
Following the pattern of creating environments that hill-climb benchmarks, Tmax (Iverson et al., 2026) is the strongest recent open RL recipe for improving terminal agents across diverse domains such as SWE, sysadmin, security, data processing, and data science. Instead of starting from existing repos, a generator agent writes the task instructions, source files, fixtures (png, audio, video) for a diverse set of personas (e.g. "red-team operator crafting an evasion payload"). Correctness is checked with exact-match, fuzz-equivalence, metric thresholds, filtering of an adversarial corpus, and more.

Of course, the synergistic nature between RL environments and evals can be abused by creating tasks that closely match the benchmark's held-out samples, effectively allowing training on the test set and inflating results. As a way forward, Florian Brand (@xeophon) has proposed to either fully embrace this nature and utilize procedural open-source RL environments as benchmarks for improving performance, or turn to fully-closed internal evals that cannot be overfitted:

I think semi-private evals and evals with a hold out set are basically dead. Data vendors will just build synth envs to hillclimb.

There's only fully open source (eg MazeBench) or internal (eg Cursorbench).

Going down the fully open-source route, our work on Reasoning Gym (Stojanovski et al., 2025) explores the idea of using procedural data generators for creating effectively infinite RLVR training data with adjustable complexity across several domains such as algebra, arithmetic, computation, cognition, geometry, graph theory, logic, and various common games. While simple in nature, the diversity of the data has already proven useful at scale, as it has been used in the training mixes of Ai2's Olmo 3 and NVIDIA's Nemotron 3 Super. The library for procedurally generating the data is open-source.

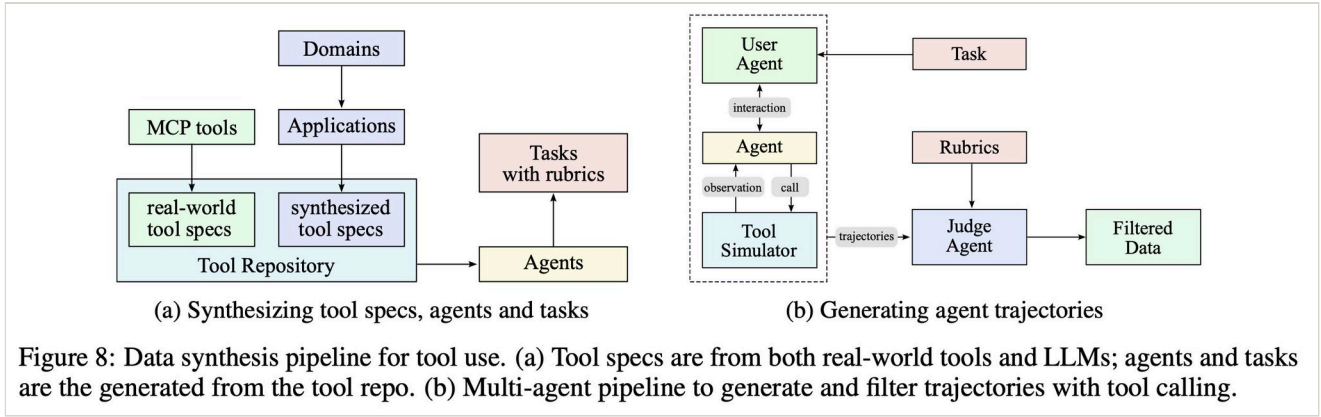


But to do any meaningful work, agents need to go beyond simple puzzles, and learn how to interact with the digital world by calling tools. This requirement has resulted in the development of RL environments for tool-calling.

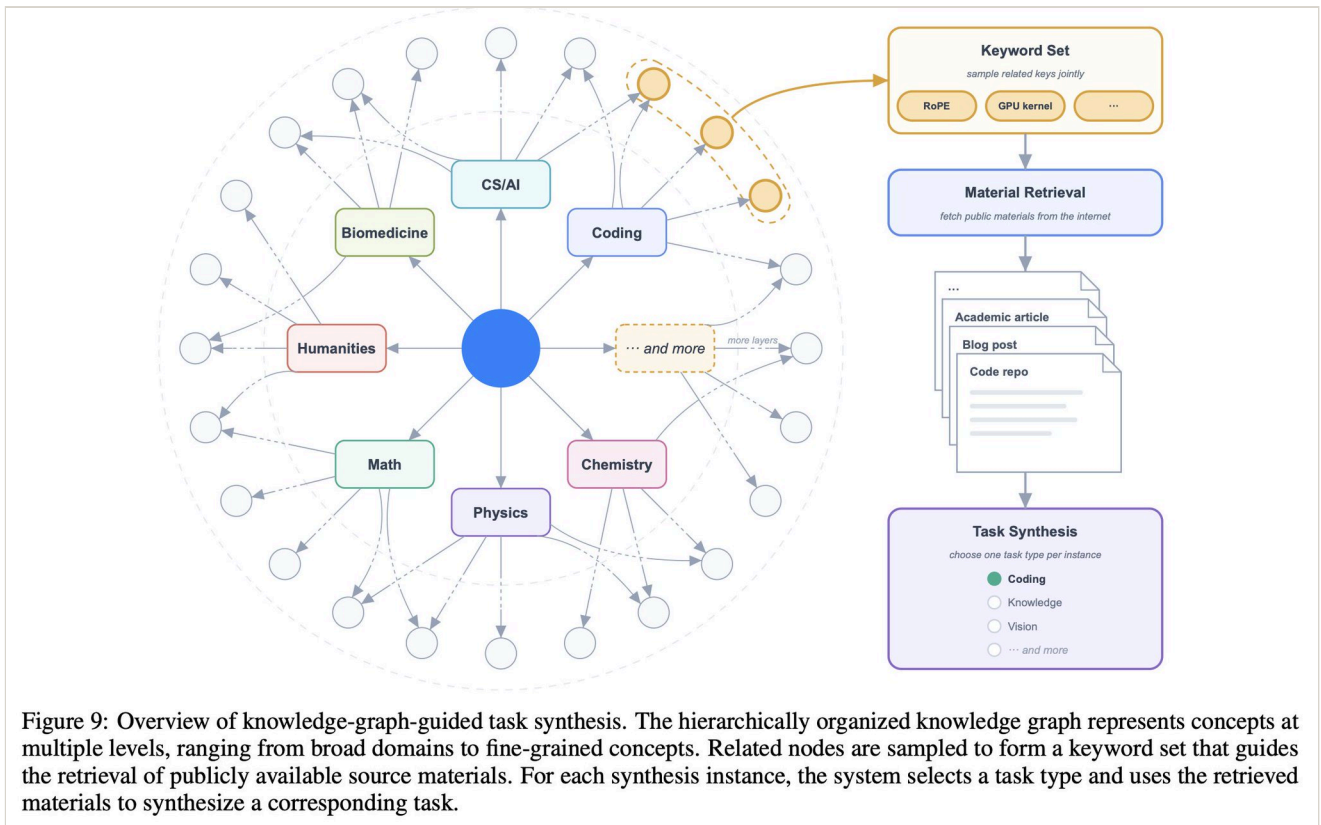
FunReason-MT (Xu et al., 2025) creates synthetic multi-turn tool-use data by constructing a Directed Acyclic Graph of tools. Given a target tool which should be reached, a combination of greedy and random sampling is utilized to create a trace by traversing the graph (e.g. `get_zipcode` → `buy_tickets` → `send_receipt`). An LLM reads this raw trace and synthesizes a plausible high level goal (e.g. "buy two tickets in Boston tonight, mail me the proof"). Given the task, an agent interacts with the environment, with a critic model steering it towards the goal. After rejection sampling, a model is trained via SFT on the correct traces.

Kimi K2 (Kimi Team et al., 2025) takes the synthetic tool-use data generation one step further. First, they fetch 3000+ real MCP tools from GitHub and further synthesize tool schemas for several diverse domains and downstream applications (finance, healthcare, robotics). Then, they instantiate various agents equipped with different subsets of tools and system prompts characterizing their area of expertise, behavior, and traits. For each agent a high-level task is created along with a rubric defining the success criteria.

The agents then generate trajectories by invoking tools in order to accomplish their respective task, where the environment feedback is mimicked either by LLM agents simulating users with distinct personas (the setup τ -bench (Yao et al., 2024) introduced for evaluating agents on stateful tool use with a simulated user), or by sophisticated tool simulators that maintain state and provide highly plausible responses. After performing rejection sampling based on the previously-defined rubrics, the traces that pass are used for SFT.

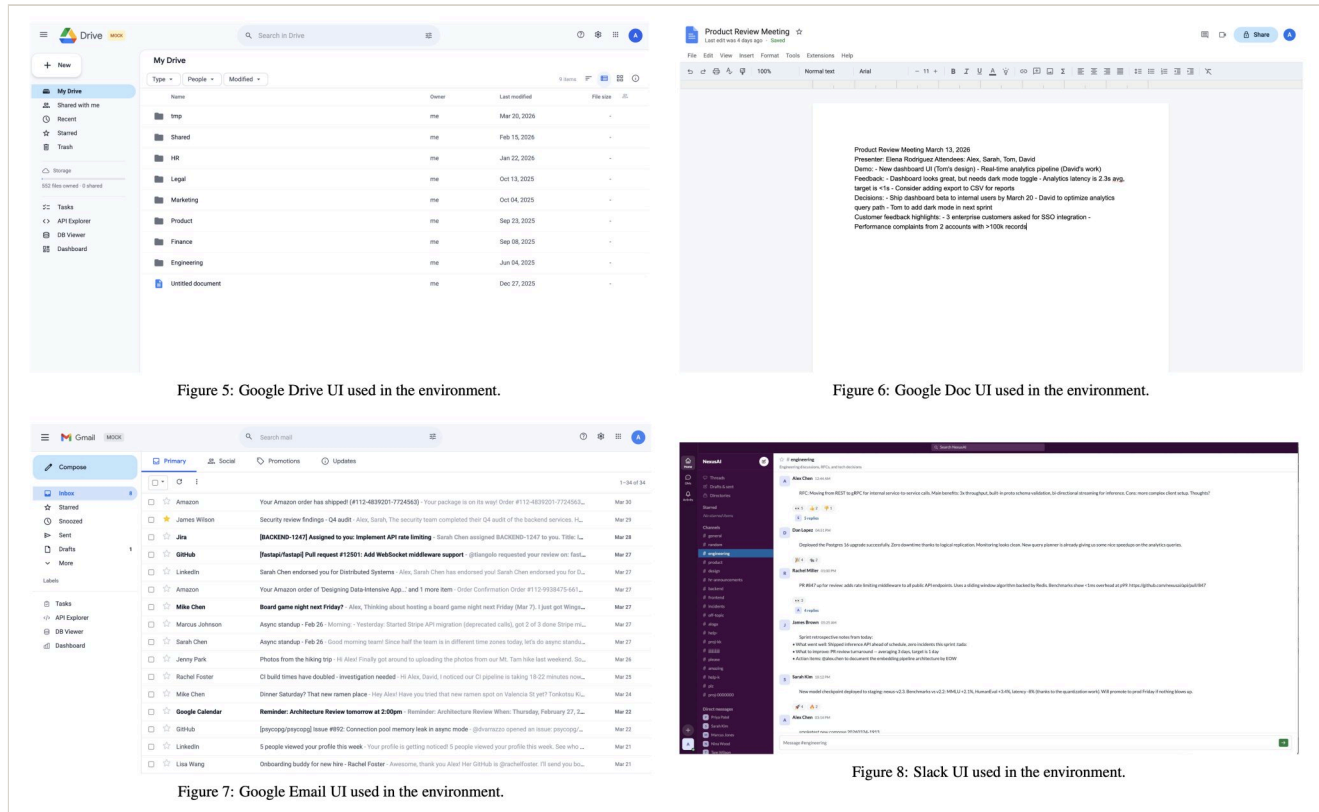


To further broaden the domain coverage of tasks, Kimi K3 (Kimi Team et al., 2026) builds an evolving hierarchical knowledge graph that agents expand through web-scale exploration. This DAG is first seeded with a predefined set of coarse categories such as Computer Science, Math, Chemistry, Physics, Biomedicine and Humanities — which agents then sample from, explore the web for, and refine recursively by adding related fine-grained concepts. For task construction, keywords derived from the sampled nodes, along with context of their coarser parent nodes, are used to perform web search to retrieve relevant materials, which are then assembled into a compact and well-defined training task specification.



Beyond scaling up synthetic environments for coding and deep research, there is a lot of interest (and investment) in tasks around personal productivity. Kimi K3 (Kimi Team et al., 2026) reports developing realistic mock implementations of popular productivity software such as Slack, Gmail, Notion and Canvas. These synthetic environments preserve the core semantics of the software, without carrying the burden of large-scale interaction with external APIs, rate limits, and privacy.

Given the generality of the software, it allows for simulating highly diverse scenarios from human resources, legal, finance, design, and many more. This allows the teams to simulate scenarios that can span several days of work, reaching several thousand tool calls and millions of tokens in context, and the correctness of each task can be validated either by deterministic rules or LLM-based evaluators equipped with scoring rubrics. For illustration, below are example mocks from a separate paper, ClawsBench (Li et al., 2026).



Going back to the scaling laws, a natural question that arises is: just how much more is there to be squeezed out of these RL environments before we start being bottlenecked by the other two axes, such as compute (where the US accounts for roughly 75% of the world's supercomputing capacity, compared to China's 15%) and the number of parameters that can be trained on said compute.

Recent evidence points to substantial headroom. GLM-5.3 (Z.ai Team, 2026) achieved massive improvements over the earlier GLM-5.2 checkpoint, entirely by improving the post-training. According to the documentation, the core difficulties have shifted from the modeling to the environment design, in order to learn from tasks which are verifiable, complex, and close to real work. To scale this process, they synthesize environments by employing research agents to collect patterns from real work and turn them into tasks comprised of multi-step trajectories, along with verifiers to score attempts. The verifiers' rubrics are designed without access to the reference solution, and are themselves verified by making sure they pass with the oracle solution, and fail with no-op or partial solutions.

Jie Tang (@jietang), the founder of Z.ai, shared a very interesting opinion on the scaling laws for training modern foundation models, concluding with the motivation behind GLM-5.3:

Which brings us to this release. Total parameters appear to matter up to a threshold — enough to hold the world — after which additional capability comes from scaling elsewhere: effective depth per

forward pass, and above all post-training. GLM-5.3 is our controlled experiment on that claim. Same base, same architecture, same total and activated parameters as GLM-5.2. One month of scaling long-horizon environments and RL. The gains are not marginal. Well, scaling has more than one dial. We turned the post-training one this time because it had the most slack left in it — not because the others are finished. Base model size, pretraining data, compute spent per forward pass: all of them are still on the table, and we will come back to each. What this experiment taught us is that the dials do not have to be turned together, and that the one worth turning next is rarely the one that was worth turning last. We are not done scaling. Next time, maybe mid-training, pre-training, and even more.

Or, intuited more concisely by Noam Shazeer (@NoamShazeer):

FLOPs were intelligence; parameters were knowledge

Given the immense importance of scaling post-training, combined with the bottleneck frontier labs face in developing their own RL environments, a new market emerged around the huge demand for this commodity, with Anthropic leadership reportedly discussing spending over \$1B on environments in a single year. Until recently, Scale AI was the established leader in the data foundry business, but after Meta acquired a 49% stake in it in June 2025, SemiAnalysis reported that many labs stopped contracting with them in order to avoid leaking the type and quantity of data they need for their model development efforts. In turn, this opened the door for many new entrants to penetrate the market.

Some of the more prominent players include Deeptune (acquired by Mercor in July 2026), Turing, Fleet, Vmax, Bespoke Labs, Preference Model, Veris.ai, Mechanize, Habitat, and many others. Most of these serve closed-source environments through exclusive contracts with the lab, in contrast to players like Prime Intellect, who open-source theirs through the Environments Hub built on top of the verifiers library.

Beyond just building environments, the labs have expressed massive interest in expert-derived task specifications and rubrics in order to expand their RL efforts in domains other than code and math. Those two domains are special because they are verifiable — written code can be executed in an interpreter and checked for correctness by running tests, and a solution to a math problem can be analytically compared to a reference answer or by checking the validity of a proof in a formal verification system such as Lean.

However, the same is not true for most other domains. Determining whether a contract clause is enforceable, a financial analysis is sound, or a medical assessment is thorough does not reduce to a simple string equality check. This ignited demand for expert-written rubrics used by an LLM-as-a-judge to score model rollouts, serving as the reward signal in the RL training loop. Below is an example of one such rubric:

Task

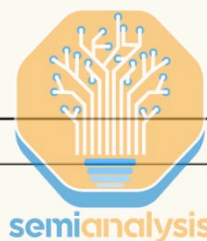
I need help analyzing Valero's refining margins in early 2024. Based on its FY2023 10-K and Q1 2024 10-Q, can you calculate the refining margin per barrel? And compare year over year?

If margins were getting squeezed while throughput volumes were consistent, figure out what drove that - is it because crude differentials were tightening up, or could it be related to how they timed their maintenance schedules? And based on the findings at that time, what would have been implied about their optimization approach heading into Q2 2024?

Rubric

Calculation Accuracy

+/-	Weight	Criterion
+	10	States FY2023 refining margin per barrel as \$17.55 (+/- \$0.05)
+	10	States FY2022 refining margin per barrel as \$21.82 (+/- \$0.05)
+	10	States Q1 2024 refining margin per barrel as \$14.07 (+/- \$0.05)
+	10	States Q1 2023 refining margin per barrel as \$22.37 (+/- \$0.05)



Attribution Analysis

+/-	Weight	Criterion
+	15	Explicitly cites Valero's Q1 2024 10-Q attribution to weaker cracks and narrower crude diffs
+	10	States that steady throughput rules out maintenance as primary driver
+	10	States that narrowing/tightening crude differentials contributed to margin compression
+	5	Notes RIN/RVO costs were lower YoY in Q1 2024 and not the margin compression driver

Source Citations

+/-	Weight	Criterion
+	10	Directly cites or references Valero FY2023 10-K filing
+	10	Directly cites or references Valero Q1 2024 10-Q filing

Q2 Optimization Strategy

+/-	Weight	Criterion
+	12	Lists at least 3 specific optimization levers (crude slate, product yield, hedging)
+	8	Recommends adjusting crude slate based on differential movements (lighter if heavy discounts tight)
+	8	Recommends maximizing gasoline/FCC runs for Q2 summer driving season strength
+	8	Recommends exploiting lower natural gas/energy costs to reduce operating expenses

Prominent players in this segment of the market are Mercor, Handshake, Surge, and Aboda.ai, with Mercor being the dominant player at about \$2B gross annualized run rate.

These expert rubrics are still predominantly written by domain experts, though there are some efforts to automate this process as well. A very cool recent example demonstrating the difficulty of designing proper rewards for non-verifiable domains is Training to Paint with Code by Surya (@kickingkeys) and team, where they teach a model with RL to create aesthetically pleasing images by writing a complete p5.brush JavaScript sketch.

The initial attempt involved a complicated reward signal combining deterministic checks

(compilation check, code-length reward), preference model, and LLM judges based on hand-curated rubric criteria (recognizability, aesthetics, technique, depth), but the policy ultimately collapsed to the same flat clip-art flower with five rounded petals, mainly because the judge criteria were highly correlated and the length term saturated. After several rounds of improvements, the final reward structure involved only a compilation check, a binary length check, a preference model, and a single LLM-as-a-judge which did pairwise comparisons between the generated image and an image from a hand-curated pool of candidates.



Nonetheless, the results were impressive and indicative of the latent challenges in designing effective rewards at scale for non-verifiable domains. In *Asymmetry of verification and verifier's law*, Jason Wei (@_jasonwei) lays out the asymmetry between the effort required in solving versus verifying a task — solving a Sudoku puzzle is hard, but validating a solved puzzle is easy; on the other end of the spectrum, claiming a particular diet as healthy is easy, but verifying the claim with scientific evidence is hard. Hence, Jason proposes the following Verifier's law:

The ease of training AI to solve a task is proportional to how verifiable the task is. All tasks that are possible to solve and easy to verify will be solved by AI.

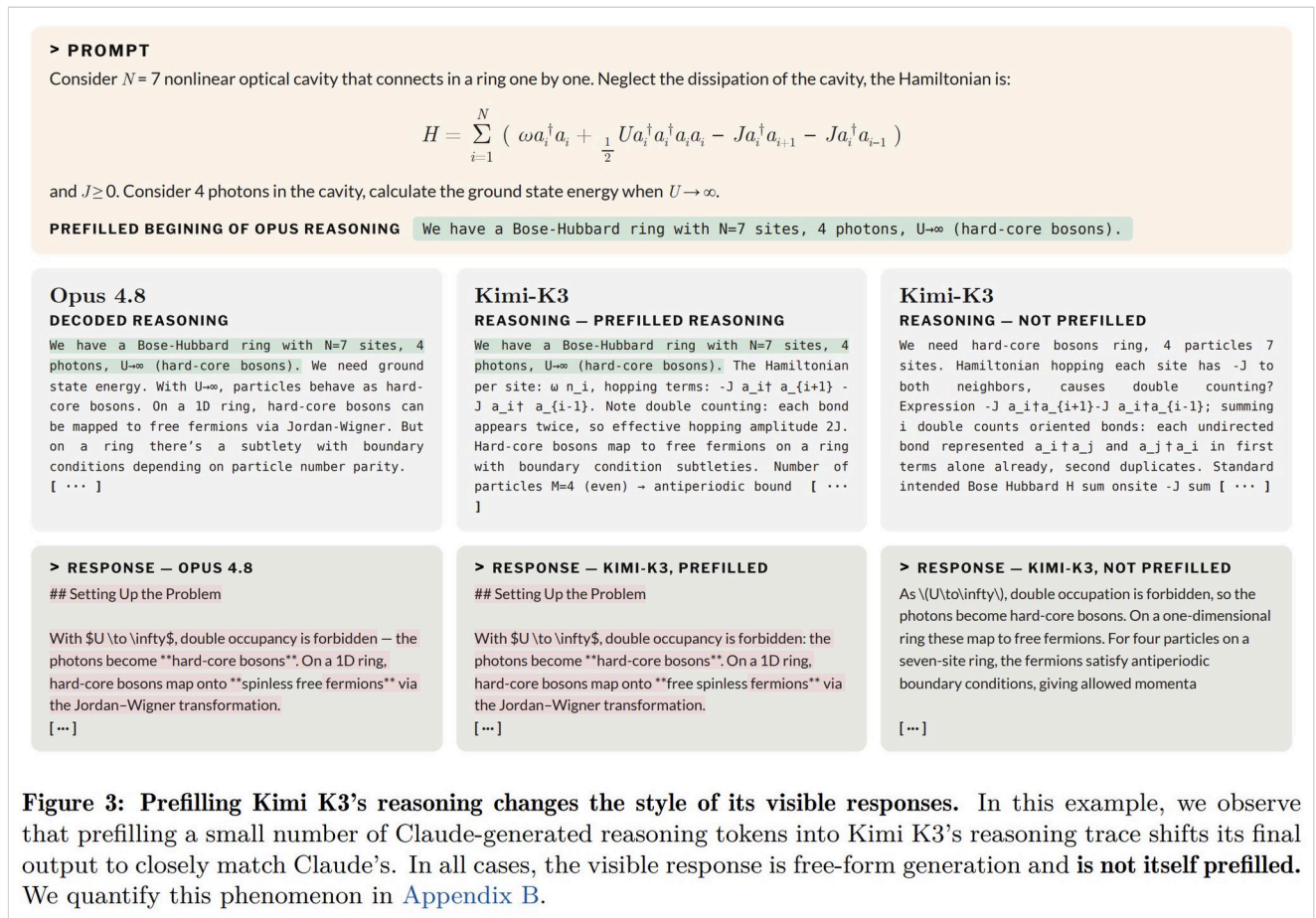
Most benchmarks used today to measure model performance satisfy this law, hence why we see saturation in only a few months after an eval is released. Nonetheless, the biggest advancements have predominantly been observed around coding and math, and a lot more challenges remain for the rest of the world's domains.

Finally, there are also huge efforts in moving RL environments from Docker containers to the real world. Periodic Labs is building closed-loop systems for automated scientific discovery, where a model proposes hypotheses, initially tests them in silico using high-fidelity simulators, and finally conducts the experiments in the physical lab.

Lately, there has been quite a lot of controversy around Chinese labs distilling frontier capabilities from US competitors at a fraction of the cost needed to learn them, starting with OpenAI's claims about DeepSeek in early 2025. Anthropic accused Moonshot AI (Kimi models) of exfiltrating 3.4 million exchanges on topics surrounding agentic reasoning, coding, and computer use.

Alexander Panfilov (@kotekjedi_ml) and team have recently publicly disclosed a vulnerability involving Stealing Reasoning Traces from Proprietary LLM APIs, including that of Anthropic, which

hinges on the architectural decision that conversation traces are encrypted and sent back to the client, being fully interchangeable between sessions, users, and models from a provider's ecosystem. By passing the encrypted reasoning traces of larger models to a weaker less safeguarded model from the same provider, they can force it to repeat the full conversation (including the hidden reasoning traces) verbatim — thereby demonstrating how a third party can curate a large reasoning dataset at scale in order to distill various model capabilities. Intriguingly, they also show suggestive (though, by their own account, inconclusive) evidence that when Kimi K3 is prefilled with a short fragment of decoded Opus 4.8 reasoning, its output shifts stylistically towards Claude and its n-gram overlap with Opus's answer increases.



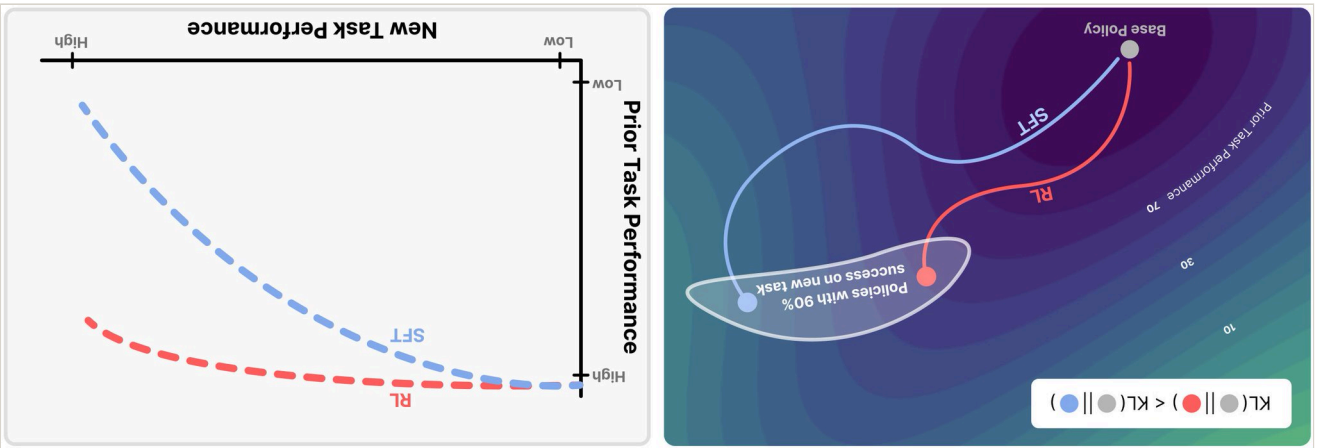
One possibility is that both Moonshot and Anthropic have bought environments from the same data vendor. It has been reported that US data shops supplying the likes of OpenAI and Anthropic, such as Surge, also sell to China's top labs.

But maybe Moonshot did distill traces from Anthropic. So, does this suggest that no lab should bother post-training and paying for RL environments when they can just distill the capabilities by SFT-ing on exfiltrated traces?

First of all, while it is certainly possible that Chinese labs have extracted traces from frontier US labs, they have at most used them in mid-training, or at the beginning of post-training as the cold-start phase of RL, to teach an instruction-tuned model the atomic building blocks of reasoning such as planning, decomposition, self-verification, and backtracking.

RL's Razor: Why Online Reinforcement Learning Forgets Less (Shenfeld et al., 2025) finds that SFT,

which is used for distilling traces from stronger models, leads to increased catastrophic forgetting of previously accumulated knowledge compared to RL due to the off-policy nature of optimization. In general, RL optimization implicitly converges to solutions which are closer to the base policy in terms of KL divergence, leading to less forgetting. Relatedly, Retaining by Doing (Chen et al., 2025) found that because SFT is minimizing forward KL as opposed to RL which is minimizing reverse KL, this results in better retention and generalization when learning from on-policy samples instead of distilling someone else's traces. All in all, this suggests that if there is some distillation happening, it's probably in the mid-training phase in order to help cold-start the RL training, as opposed to the post-training phase.



To date, Kimi K3 has served almost 2T tokens through OpenRouter alone, in line with the broader shift of open-model usage toward Chinese labs, with top 5 usage coming from Hermes Agent, Claude Code, pi, OpenHands, and Cline. These trends imply that the model has been used for frontier-level work, including specialized, complex, and long-horizon tasks. Most of these are out-of-distribution challenges which the model hasn't encountered during training, meaning that it must have very strong generalization capabilities.

SFT Memorizes, RL Generalizes (Chu et al., 2025) shows that SFT tends to memorize the training data and does not generalize out-of-domain, while RL with outcome-based rewards generalizes to unseen tasks much better. Since outcome rewards can only be obtained by sampling from the model's own policy, the model learns from its own experience and mistakes rather than imitating someone else's. In line with this, Generalized Knowledge Distillation (GKD) (Agarwal et al., 2023) highlights that a model which has been optimized off-policy never samples from its own policy, which at inference time can drift away from the training distribution it imitated, resulting in catastrophic errors that compound dramatically. Today's agentic use-cases are an extreme version of long-horizon tasks, with rollouts that can span hundreds of thousands of tokens, implying that the model must possess the capabilities to self-reflect and backtrack when needed.

For these reasons, distillation can only get you so far, and is not some silver bullet capable of circumventing the expensive and challenging post-training phase in order to reach the frontier of model capabilities. In fact, it is absolutely necessary to own your environments, sample outputs from your own model, and learn from your own experience. Nathan Lambert (@natolambert) concisely sums it up in How Chinese labs keep stride with the frontier on Interconnects:

One does not simply “distill” RL environments, infrastructure to run them at scale, or algorithms to mix them together effectively.

Conclusion

From the outside, frontier model capabilities seem to recursively improve along an exponential with no visible horizon. In my opinion, that impression is mostly an artifact of just how little these labs disclose the recipes of their pipelines. Open recipes for scaling training are sparse, as those that work well are generally well-kept secrets at frontier labs. This article was my attempt to improve that transparency, built entirely on what the community published openly.

Hopefully, I have convinced you that RSI could stand for **Recursive Synthetic Improvement**: one generation of models improving the next by synthetically augmenting the data throughout the entire stack — the Judge, the Corpus, the Teacher, the Curriculum, and the Environment. And if early experiments in pushing the frontier of science hold, we might even see the same happen to the Researcher.

Whether this paradigm is a major contributing factor that widens the gap between open and closed-source models, keeping them on different exponentials, or a driving force for open-source to cheaply and effectively catch up, is yet to be determined.

